Memetic Computing (2025) 17:11
https://doi.org/10.1007/s12293-025-00448-4

METHODOLOGY

Multifactorial memetic algorithm with adaptive auxiliary tasks for service migration optimization in mobile edge computing

Guo Li[1] · Zhaobo Liu[1] · Ling Liu[2] · Zexuan Zhu[3,4]

Received: 3 April 2024 / Accepted: 2 March 2025 / Published online: 21 March 2025

Abstract
In high-speed mobile networks, mobile edge computing is tasked with service migration optimization, i.e., assigning mobile users to the right servers to minimize the response time. Service migration optimization is a complex problem posing significant challenges to conventional optimization methods. To tackle this problem, we develop a multifactorial memetic algorithm with adaptive auxiliary tasks or MFMA-AAT for short. MFMA-AAT solves the target service migration optimization problem and an adaptively selected auxiliary task simultaneously, where the auxiliary task is a simplified version of the target problem to guide the search towards promising regions faster via knowledge transfer. Multiple auxiliary tasks are pre-constructed based on the distribution of the mobile users and the one with the best improvement at each generation is selected for knowledge transfer. A community detection-based memetic operator is also introduced to accelerate the local convergence of the proposed algorithm. Experimental results on test problems demonstrate that MFMA-AAT is more efficient than traditional service migration approaches and other state-of-the-art multifactorial evolutionary algorithms.

Keywords Evolutionary multitasking · Multifactorial evolutionary algorithm · Multi-form evolutionary optimization · Service migration optimization · Mobile edge computing

1 Introduction

With the exponential growth of connected end devices and the constant emergence of applications in mobile networks, delivering a user-centric service is becoming increasingly crucial [1]. The long physical transmission distance can impede the reduction of round-trip delay [2], making traditional cloud computing insufficient for providing low-latency service. To address this issue, mobile edge computing (MEC) steps in, bringing computing resources closer to the mobile user equipment (UE) and reducing response time [3]. MEC is not just another form of cloud computing on distributed edge servers, like cloudlet and fog computing [4, 5]. It is latency-aware for dynamically allocating and adjusting resources including CPU, storage, and bandwidth to prevent MEC service degradation for the UE [6]. With the server being located close to the base station and UE, MEC networks offer ultra-low response times [7].

Intuitively, it seems like keeping the UE service profile close to the UE by placing it on the nearest MEC server and following the UE's movements would lead to the lowest latency. However, the actual user-perceived latency is mainly impacted by communication, migration, and calculation latencies. Current profile tracking algorithms aim to minimize the communication latency [8], but this optimization comes with a trade-off in terms of migration latency. In addition, migration can lead to hotspot congestion on MEC

✉ Zexuan Zhu
zhuzx@szu.edu.cn

Guo Li
szuliguo@szu.edu.cn

Zhaobo Liu
liuzhaobo@szu.edu.cn

Ling Liu
liuling@xidian.edu.cn

[1] Shenzhen City Key Laboratory of Embedded System Design College of Computer Science and Software Engineering, Shenzhen University, Shenzhen 518060, China

[2] Guangzhou Institute of Technology, Xidian University, Guangzhou 510530, China

[3] Key Laboratory of Dependable Service Computing in Cyber Physical Society, Ministry of Education of China, Chongqing University, Chongqing 400044, China

[4] National Engineering Laboratory for Big Data System Computing Technology, Shenzhen University, Shenzhen 518060, China

servers, further increasing calculation latency [9]. This makes service migration of mobile computing capabilities a crucial area of research [10]. Service migration needs to tackle the challenge of selecting the right MEC server among a pool of candidates to place the UE profile for an ongoing service [11].

Traditional service migration algorithms are mainly based on game theory (GT) [12] that tends to get trapped in local optima. Recently, evolutionary algorithms (EAs) have been employed in the optimization of service migration thanks to their strong global search capability [13]. Nevertheless, traditional EAs solve one single service migration optimization problem at a time from scratch, which might suffer from slow convergence as the scale and complexity of the target problem increase. Evolutionary multitasking (EMT) [14–17] can serve as a candidate solution to this issue by solving the target service migration optimization task with additional auxiliary task(s) simultaneously, which is also known as multi-form evolutionary optimization [18]. An auxiliary task usually is a related task or a simplified version of the original target problem. Solving the auxiliary task can provide helpful knowledge to assist the solve of the target problem in EMT via knowledge transfer. For example, multifactorial evolutionary algorithms (MFEAs) [14, 19] represent the first attempts to solve multiple related optimization tasks simultaneously. They have been demonstrated to achieve better performance than their single-task counterparts. Based on MFEA framework, MFEA-MVD [20] employs multiobjectivization via decomposition to generate effective auxiliary tasks to improve the solving of the primary task. Shang et al. [21] solved vehicle routing problems by introducing simplified auxiliary tasks in EMT framework, showing that knowledge transfer between tasks can improve the evolutionary search. Yang et al. [22] solved operational indices optimization in beneficiation processes with multi-objective EMT algorithm by constructing less-accurate models as auxiliary tasks to enhance the convergence of the most accurate model.

The effectiveness of the constructed auxiliary task(s) is pivotal to the success of multi-form EMT. Many EMT-based algorithms use constant auxiliary task throughout the evolutionary search, which might be less efficient. Using different auxiliary tasks in different stages of the search could be more beneficial to solve the target problem [23–26]. To this end, this work introduces MFMA-AAT, a multifactorial memetic algorithm with adapts auxiliary tasks (AAT) to solve service migration optimization problem in MEC. By using K-means clustering, MFMA-AAT initially groups UEs based on their locations in MEC and constructs auxiliary tasks corresponding to these clusters to optimize resource allocation in each group. In each iteration, MFMA-AAT dynamically assigns a higher selection probability to the auxiliary task that contributes most to the convergence of the primary task. MFMA-AAT also imposes a memetic operator based on community detection and local search to accelerate the local convergence. Particularly, using cosine similarity in community detection [27], MFMA-AAT pre-selects a few MEC servers as a neighborhood for a candidate solution and then uses hill climbing to find the local optimum within the neighborhood.

MFMA-AAT boosts the efficiency of optimization with dynamically selected auxiliary tasks that enhance the search with adaptive knowledge transfer. The convergence of the primary task is accelerated and the search space is compacted via the memetic operator. Empirical evaluations on test problems show that MFMA-AAT outperforms the traditional GT methods and other state-of-the-art MFEAs in identifying the optimal service migration solution. The innovation of MFMA-AAT lies in its ability to construct auxiliary tasks without prior knowledge, leveraging the intrinsic properties of the data to dynamically design efficient positive transfer auxiliary tasks that adapt to the data. The contributions of this study are summarized as follows:

- Service migration optimization in MEC is for the first time formulated as a multi-form optimization problem, enabling EMT to solve the problem efficiently via knowledge transfer.
- An adaptive selection mechanism of auxiliary task is explored in multi-form EMT to solve service migration optimization problem. The selection of auxiliary task considers spatiotemporal characteristics of MEC networks, where K-means clustering of UEs is introduced to constructed auxiliary tasks that are then selected based on their contributions to the optimization of the primary task.
- A memetic operator is designed to accelerate the convergence of the algorithm in local regions by leveraging community detection and hill climbing local search.
- The resultant algorithm MFMA-AAT is extensively investigated in different scales of service migration optimization problems to reveal the strength and weakness.

The remainder of this paper is organized as follows: Sect. 2 reviews the related work on the subject. Section 3 provides the preliminaries and problem formulation. Section 4 details the proposed MFMA-AAT. The experimental results are described in Sect. 5. Finally, Sect. 6 summarizes the main conclusions of this study.

2 Related work

This section provides an overview on the related work of EA-based MEC and EMT algorithms.

2.1 EA-based MEC

EAs have been widely used in MEC scheduling. For example, Zhu et al. [28] proposed an edge-computing resource allocation strategy based on an improved genetic algorithm. The paper adopted integer coding, knowledge-based crossover, and mutation of population segmentation to improve the optimization ability of the genetic algorithm. Bi et al. [29] formulated a genetically simulated annealing-based particle swarm optimization to achieve joint optimization of computation offloading between a cloud data center and edge server. Ahmed et al. [30] considered parallel and sequential task offloading for multiple MEC servers. Based on the genetic algorithm, simulation results demonstrate that the sequential solution provides a dropped failure probability, and the parallel solution yields a lower latency.

Liu et al. [31] presented a joint optimization objective of limited resources, higher operating costs, and certain failure probabilities in MEC, and subsequently introduced an optimized task allocation approach with a biogeography-based optimization algorithm, which is an evolutionary algorithm that can maximize the effectiveness of task allocation and reduce communication delay simultaneously. Tang et al. [32] proposed a novel offloading algorithm based on a genetic algorithm for finding the optimal offloading decision in MEC, which not only optimizes the tasks in the current network slice but also reduces the task pressure in the next network slice, which can better exploit the computing capacities of the MEC servers. Although valuable results have been achieved in the optimization of MEC performance using EAs, service migration remains an immature and highly unproven technology.

2.2 EMT algorithms

The first EMT paradigm was first proposed in [14] as MFEA, where EMT was shown to outperform single-task EAs in solving multiple optimization problems simultaneously thanks to the introduction of knowledge transfer. The efficiency of knowledge transfer is critical to the performance of EMT algorithms. For example, Bali et al. [19] improved MFEA to MFEA-II by leveraging online learning and task similarities to improve the efficiency of knowledge transfer. Wei et al. [33] concluded that traditional EAs might suffer from a high computational burden and poor generalization ability in complex optimization problems, and MFEA is a candidate solution to overcoming the problem by efficient knowledge extraction across distinct optimization task domains. Zhou et al. [34] recognized that the crossover is the basis for achieving knowledge transfer in MFEA, and different crossover operators affect efficiency of knowledge transfer. Thus, they proposed MFEA-AKT to update crossover dynamically for adaptive knowledge transfer.

Xu et al. [35] introduced an MFEA to solve multiple optimization tasks and avoid negative knowledge transfer between tasks. They proposed a series of adaptive knowledge transfer operators to make the best use of knowledge transfer. Liang et al. [36] considered that knowledge transfer is easily trapped in a local optimum when multitasks are highly similar and suffers from negative transfer when multitasks have low similarity. Hence, they proposed an adaptive knowledge transfer method based on the population distribution. Feng et al. [37] found that most EMT algorithms exchange knowledge implicitly by crossover, so they presented an EMT algorithm based on explicit knowledge transfer among multiple tasks. The experimental results on vehicle routing benchmarks showed the efficacy of explicit knowledge transfer based on the solution.

Gao et al. [38] introduced subspace distribution alignment and decision variable transfer mechanisms into multiobjective EMT to promote the positive knowledge transfer across tasks. Tang et al. [39] devised a level-based intertask learning strategy to transfer sharing information among the cross-task neighborhood for multi-task particle swarm optimization with a dynamic local topology. Jiang et al. [40] proposed block-level knowledge transfer algorithm for EMT, which divides each individual into multiple gene blocks of equal size and then groups the blocks using K-means clustering. Crossover and mutation operators are performed on the block population within the same group to facilitate positive knowledge transfer. Li et al. [41] used an adaptive information transfer mechanism to solve a competitive multitasking optimization problem in which all task objectives are comparable.

EMT facilitates the simultaneous convergence of multiple tasks. In practical scenarios, prioritizing a single task as the primary focus while relegating the others to an auxiliary role can significantly enhance the acceleration of the target task.

2.3 EMT algorithms with auxiliary tasks

A few multi-form EMT algorithms were proposed to speed up the convergence a primary optimization task by introducing auxiliary tasks with positive knowledge transfer.

For instance, some EMT algorithms construct auxiliary tasks by decomposing the main task into smaller subtasks. Ma et al. [20] proposed MFEA with multiobjectivization via decomposition to construct positive related auxiliary tasks. Its auxiliary tasks are constructed through subtask decomposition, which required certain prior knowledge and appropriate combination of subtasks. Liu et al. [42] incorporated surrogate-assisted evolutionary algorithms for optimization problems with a costly fitness evaluation. Via parallel random grouping for the costly problem, the assisted subtasks as auxiliary tasks with constraint boundary can speed up the convergence. Yang et al. [43] applied EMT framework to

solve the costly task offloading problem in MEC networks, where subproblems of the target problem were constructed as auxiliary tasks to help the solving of the target problem. Qiao et al. [44] solve constrained multi-objective optimization problems (MOPs) with auxiliary MOPs extracted from the target problems using EMT. Wang et al. [45] employed surrogate-assisted MFEA to address the challenges of a high-dimensional search space and high computation cost optimization. A surrogate model in the joint space of the decision and scenario spaces was constructed to replace part of the expensive function evaluations.

Furthermore, certain EMT algorithms develop auxiliary tasks that are similar to the main task by leveraging constrained conditions or prior knowledge. Qiao et al. [23] transformed the constrained multiobjective optimization problem into several simple tasks with dynamic constraint boundaries as auxiliary tasks, maintaining a high degree of relevance with the main task, thereby providing supplementary evolution directions. Qiao et al. [24] introduced a method for constructing global and local auxiliary tasks. The global auxiliary task facilitates global search by disregarding constraints, while the local auxiliary task enhances regional diversity around the population of the main task. Ming et al. [25] adjusted the auxiliary tasks flexibly according to current optimization needs and constraint conditions to improve search efficiency for constrained multi-objective optimization problems. Yang et al. [22] constructed less-accurate models as auxiliary tasks to solve the operational indices optimization in beneficiation processes within multi-objective EMT framework. Yang et al. [46] proposed an effective EMT which incorporates segmented knowledge transfer via cheap and simple auxiliary tasks, with the flexibility to dynamically update these auxiliary tasks during the optimization process. Chen et al. [47] constructed an auxiliary task similar to the main task and subsequently adaptively adjusted the knowledge transfer probability values of the auxiliary task for each generation. Shang et al. [21] used a memetic search with EMT to solve the vehicle routing problem, which was simultaneously performed on multiple simplified vehicle routing problems as auxiliary tasks and the original VRP as the main task, and each task will perform a local search.

It is noted that the existing multi-form EMT algorithms that use constant auxiliary task during all stages of the search might result in inefficient knowledge transfer [23, 24]. Selecting auxiliary tasks adaptively according to the staged search performance of the algorithm could be more favorable. In this regard, we propose the MFMA-AAT as follows.

3 Problem formulation

Before introducing the proposed algorithm, in this section we provide the preliminaries and problem formulation of the service migration optimization problem in MEC network. For the convenience of the reader, the notations adopted in this paper are summarized in Table 1.

in MEC network, as depicted in Fig. 1, a UE can move between base stations and the UE's service profile is migrated between MEC servers. Each UE profile is restricted to only one MEC server during a fixed time slot. In each time slot, the UE's location is recorded only once, and the UE's service profile remains unchanged on an MEC server. Let an MEC network contain η UEs, i.e., U=$\{u_1, u_2, \ldots, u_i, \ldots, u_\eta\}$ and ω base stations, where $U_i \in \{1, ..., \omega\}$ indicates the index of base station on which the i-th UE is allocated in the current time slot. For the sake of simplicity, each base station is paired with only one MEC server. Each UE has a one-to-one corresponding UE profile P=$\{p_1, p_2, \ldots, p_i, \ldots, p_\eta\}$ with $p_i \in \{1, ..., \omega\}$ indicating the MEC server index where the i-th UE profiled is allocated in the current time slot. In an MEC system, the user-perceived latency is decided by the interplay between three latency factors of communication, migration, and computation.

(1) The communication latency l^α is comprised of two components. One is the radio propagation latency from the base station to the UE, which is assumed a constant σ. The other component is the transmission latency between the MEC server and the base station. The transmission latency is determined by the number of hops from the profile-hosting MEC server to the server belonging to the UE associated base station. According to [8], we assume that the UE profile transmission latency between neighboring base stations is equal to one hop. The radio propagation latency σ is set to half the hop time. In our experiments, the hop time is taken as the basic unit of the user-perceived latency. In the MEC network shown in Fig. 1, the corresponding hop times between the MEC servers in the network are listed in Table 2. The communication latency l^α of u_i is defined as follows:

$$l_i^\alpha = \mathrm{Hop}(u_i, p_i) + \sigma \tag{1}$$

where u_i and p_i indicate the indexes of allocated base station and served MEC server of u_i in the current time slot, respectively, and the function $\mathrm{Hop}(\cdot, \cdot)$ denotes the transmission latency of two locations. As the indices of the base stations and MEC servers are the same, the transmission latency can be quantified as the hop time reported in Table 2.

(2) The migration latency l^β occurs when a UE profile migrates to a new MEC server. Migration latency is proportional to the migration distance, namely, the number of hops between two MEC servers. Thus, the migration latency l^β of u_i is defined as follows:

$$l_i^\beta = \mathrm{Hop}(p_i, p_i') \tag{2}$$

Table 1 Main notations used in this paper

Symbol	Description
η	Number of UE
ω	Number of MEC servers
U	User equipment connected to base stations $\{u_1, u_2, ..., u_i, ..., u_\eta\}$
P	UE profile set $\{p_1, p_2, ..., p_i, ..., p_\eta\}$
M	MEC server set $\{m_1, m_2, ..., m_j, ..., m_\omega\}$
κ	Number of K-means groups
G	K-means group set $\{\mathbf{g}_1, \mathbf{g}_2, ..., \mathbf{g}_k, ..., \mathbf{g}_\kappa\}$
$\mathbf{g}_k$	A 0-1 array mask of length η
τ_j	The allocated computation capability to each UE on server j under an average allocation strategy
r_i	Computation requirement of u_i
q_i	Queued number of required calculations by u_i
l_i^α	Communication latency of u_i
l_i^β	Migration latency of u_i
l_i^γ	Computation latency of u_i
l_i	User-perceived latency of u_i
$\mathbf{x}$	A migration solution for all profiles, also a chromosome
Ω	The solution space
$\ell(\mathbf{x})$	The average user-perceived latency in the MEC system
λ	Number of chromosomes in a population
$\mathrm{p^c}$	Probability of crossover
$\mathrm{p^m}$	Probability of mutation
$\mathrm{p^l}$	Probability of local search
X	A population, also a solution set $\{\mathbf{x_1}, \mathbf{x}_2, ..., \mathbf{x}_v, ..., \mathbf{x}_\lambda\}$
c_i^v	The i-th gene in the chromosome $\mathbf{x}_v$
$f_0(\mathbf{x})$	Main task objective function, $f_0(\mathbf{x})$=$l(\mathbf{x})$
$f_k(\mathbf{x})(k>0)$	Auxiliary task objective function, $f_k(\mathbf{x})$=$l(\mathbf{x}.*\mathbf{g}_\mathrm{k})$
$\boldsymbol{\epsilon}$	Select probability array on κ auxiliary tasks $[\epsilon_1, \epsilon_2, ..., \epsilon_k, ..., \epsilon_\kappa]$, $\sum_1^\kappa \epsilon_k$=1
Run^G	Number of generations
Max^E	Maximum number of evaluations
X^n	The population in the n-th iteration, $n \in [1, \mathrm{Run}^G]$
δ_k^n	Drop rate of auxiliary task k in the n-th iteration
$\boldsymbol{\xi}_i$	A practical attribute vector of p_i
$\boldsymbol{\xi}_i^\star$	An ideal attribute vector of p_i
$\boldsymbol{\phi}_i$	A MEC server community of u_i

where p_i and p_i' denote the indices of the MEC servers where u_i is allocated in the current and previous time slots, respectively.

(3) The computation latency l^γ of a UE's requirements on an MEC server is based on the amount of computation needed and the resources allocated to the UE by the MEC server. We define the computation requirement of u_i in the current time slot as r_i. Under an average allocation strategy, the allocated resources to each UE on server p_i are determined by dividing the computing capacity of server p_i by the number of UEs connected to server p_i in the current time slot, which is denoted as τ_i. The computation latency of u_i is defined as follows:

$$\begin{aligned} l_i^\gamma &= \frac{r_i + q_i'}{\tau_i}, \\ q_i &= \max(q_i' + r_i - \tau_i,\ 0), \end{aligned} \tag{3}$$

where q_i' indicates the computation requirements of u_i that remain from the previous time slot. There is no remaining computation in the current time slot q_i, i.e., $q_i = 0$, if and only if the allocated computation resources τ_i exceed the current requirements r_i plus the remaining computation from last time slot q_i'.

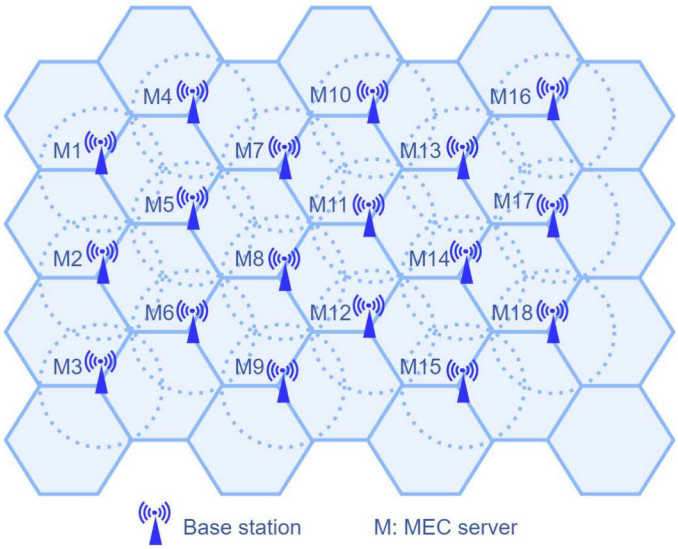


Fig. 1 Illustration of a typical MEC network. Each base station is allocated with an MEC server

Table 2 Hop times between MEC servers

	m_1	m_2	m_3	m_4	m_5	m_6	m_7	m_8	m_9	m_{10}	m_{11}	m_{12}	m_{13}	m_{14}	m_{15}	m_{16}	m_{17}	m_{18}
m_1	0	1	2	3	1	2	3	2	2	3	4	3	3	4	4	4	4	5
m_2	1	0	1	2	1	1	2	2	2	2	3	3	3	3	4	4	4	4
m_3	2	1	0	1	2	1	1	3	2	2	2	3	3	3	4	4	4	4
m_4	3	2	1	0	3	2	1	4	3	2	2	4	3	3	5	4	4	4
m_5	1	1	2	3	0	1	2	1	1	2	2	2	2	3	3	3	3	4
m_6	2	1	1	2	1	0	1	2	1	1	2	2	2	2	3	3	3	3
m_7	3	2	1	1	2	1	0	3	2	1	1	3	2	2	4	3	3	3
m_8	2	2	3	4	1	2	3	0	1	2	3	1	2	3	2	2	3	4
m_9	2	2	2	3	1	1	2	1	0	1	2	1	1	2	2	2	2	3
m_{10}	3	2	2	2	2	1	1	2	1	0	1	2	1	1	3	2	2	2
m_{11}	4	3	2	2	2	2	1	3	2	1	0	3	2	1	4	3	2	2
m_{12}	3	3	3	4	2	2	3	1	1	2	3	0	1	2	1	1	2	3
m_{13}	3	3	3	3	2	2	2	2	1	1	2	1	0	1	2	1	1	2
m_{14}	4	3	3	3	3	2	2	3	2	1	1	2	1	0	3	2	1	1
m_{15}	4	4	4	5	3	3	4	2	2	3	4	1	2	3	0	1	2	3
m_{16}	4	4	4	4	3	3	3	2	2	2	3	1	1	2	1	0	1	2
m_{17}	4	4	4	4	3	3	3	3	2	2	2	2	1	1	2	1	0	1
m_{18}	5	4	4	4	4	3	3	4	3	2	2	3	2	1	3	2	1	0

Overall, the user-perceived latency of u_i in the current time slot is given by:

$$l_i = l_i^{\alpha} + l_i^{\beta} + l_i^{\gamma}, \tag{4}$$

Given a group of UEs U=$\{u_1, u_2, \ldots, u_i, \ldots, u_\eta\}$ in the current time slot, the goal is to optimize the corresponding UE profiles migration solution $\mathbf{x}=[p_1, p_2, \ldots, p_i, \ldots, p_\eta]$ such that the average user-perceived latency of all UEs is minimized, i.e.,

$$\min \ell(\mathbf{x}) = \frac{1}{\eta} \sum_{i=1}^{\eta} l_i \tag{5}$$

where $\mathbf{x}$ is the vector of decision variables representing the profile placement of all UEs in the current time slot.

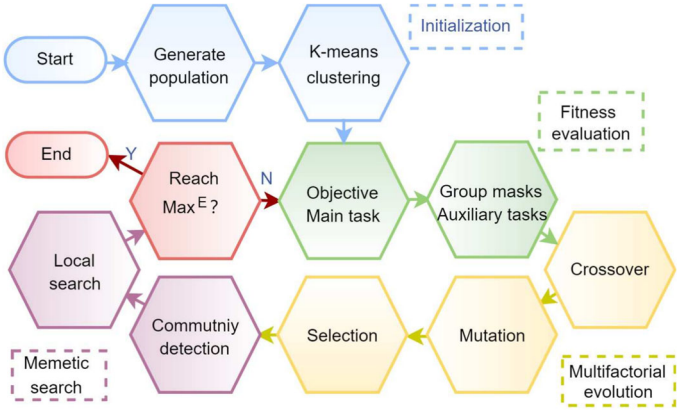


Fig. 2 Flowchart of MFMA-AAT algorithm. (i) Initialization: generate population and group UEs with K-means. (ii) Evaluation: set the main task's objective, get masks of groups according to the UE index, and set auxiliary tasks corresponding to the masks. (iii) multifactorial evolution: crossover and mutate chromosomes, compute the factorial cost and get factorial rank, select elite chromosomes. (iv) Memetic search: perform community detection to generate a neighboring space, and conduct a local search to provide an enhanced population. (v) Terminate condition: the maximum fitness evaluation Max^E

Algorithm 1 MFMA-AAT

Input: Population size η, Number of K-means groups κ, Probability of crossover p^c, Probability of mutation p^m, Objective function $f_0(\mathbf{x})$, Terminate conditions: Max^E

Output: The optimal solution $\mathbf{x}^\star$

Initialize population X: $\{\mathbf{x_1}, \mathbf{x}_2, ..., \mathbf{x}_v, ..., \mathbf{x}_\eta\}$
Evaluate X on the main task $f_0(\mathbf{x})$
Run K-means clustering to attain $\{\mathbf{g}_1, \mathbf{g}_2, ..., \mathbf{g}_k, ..., \mathbf{g}_\kappa\}$
Construct auxiliary tasks $f_k(\mathbf{x}) = \ell(\mathbf{x} .* \mathbf{g}_k)$
Initialize auxiliary selection probability $\{\epsilon_k | \epsilon_k = 1/\kappa, k \in (1, \kappa)\}$
for v = 1 to λ **do**
  Initialize skill factor of $\mathbf{x}_v$ from $[0, \kappa]$ randomly
end for
while NOT reach Max^E **do**
  $\text{X}^{offspring} \leftarrow$ grouped_uniform_crossover(X, p^c)
  $\text{X}^{offspring} \leftarrow$ swap_mutate($\text{X}^{offspring}$, p^m)
  Select auxiliary task k=roulette($\{\epsilon_k | k \in (1, \kappa)\}$)
  Set auxiliary function $f_k(\mathbf{x}) = \ell(\mathbf{x} .* \mathbf{g}_k)$
  $\text{X}^{union} \leftarrow \text{X} \cup \text{X}^{offspring}$
  for $v = 1$ to $2 * \lambda$ **do**
    if $\mathbf{x}_v.skill\_factor$==0 **then**
      $\mathbf{x}_v.factorial\_cost = f_0(\mathbf{x}_v)$
    else
      $\mathbf{x}_v.factorial\_cost = f_k(\mathbf{x}_v)$
    end if
  end for
  compute_factorial_rank(X^{union})
  compute_scalar_fitness(X^{union})
  X $\leftarrow$ select_elitist(X^{union})
  compute_drop_rate (δ_k)
  update_selection_probability($\{\epsilon_k | k \in (1, \kappa)\}$)
  X $\leftarrow$ memetic(X) (See Algorithm 2)
end while
$\mathbf{x}^\star$ = select_optimal(X)

4 Proposed MFMA-AAT

In this section, we present the details of MFMA-AAT. Figure 2 and Algorithm 1 illustrate the flowchart of the algorithm. MFMA-AAT contains four primary components namely initialization, fitness evaluation, multifactorial evolution and memetic search. Particularly, MFMA-AAT starts by randomly generating an initial population, each chromosome's cluster groups being identified based on the UE location in the MEC network. After setting the main task as classical MFEAs, the AAT method adaptively selects auxiliary task in each generation. MFMA-AAT defines a mask string to symbolize each group, with the same number of auxiliary tasks as there are clustered groups. The multifactorial evolution stage involves crossover, mutation, computation of factorial cost, scalar fitness evaluation for the population, and selection of offspring. The evolution process is then followed by a memetic operator, which performs a local search using community detection to determine the optimal individuals and update the population. The algorithm continues until the maximum fitness evaluation is reached.

Note that the chromosome generation, the clustering method for UE, the construction of the main and auxiliary tasks, as well as the corresponding fitness evaluation functions are specifically proposed for MEC systems. The AAT approach, the customized grouped uniform crossover operator, and the memetic search method that leverages community detection are put forward to ensure the efficiency of MFMA-AAT.

4.1 Initialization and evaluation functions

To initialize a solution for service migration in MEC, we start by generating a population randomly. Each individual in the population is a chromosome encompassing a group of genes. In this context, a chromosome is a η-dimensional vector of decision variables with each decision variable or

gene indicating the corresponding MEC server id where the UE is allocated. A chromosome represents a complete profile migration solution for all UEs in the MEC system, i.e., equivalent to $\mathbf{x}$ defined in Sect. 3.

The initial population is a group of solutions usually represented by a matrix X $[\eta * \lambda]$, where λ indicates the number of chromosomes and η denotes the length of a chromosome representing the number of UE. When generating an initial population, each gene value is randomly selected from $[1, ..., \omega]$, where ω denotes the number of MEC servers. The number of chromosomes should be even for the subsequent evolutionary operators.

The optimization objective of the main task is user-perceived latency, and the fitness evaluation function is given by

$$\min : f_0(\mathbf{x}) = \ell(\mathbf{x}), \qquad s.t. \quad \mathbf{x} \in \Omega, \tag{6}$$

where $\mathbf{x}$ is a decision vector of a migration solution, Ω is the solution space, and $\ell(\mathbf{x})$ is the average user-perceived latency in the MEC system as defined in Eq. (5).

For a randomly generated population, the initial skill factor is evenly distributed between the tasks. The tasks comprise the main task and all the auxiliary tasks. In a chromosome, a gene is from 1 to ω, which represents the profile of the UE belonging to the MEC server. To share chromosomes between the main and auxiliary tasks, we add a new gene, 0, which indicates that the UE is not in the MEC network. In the auxiliary task, the evolutionary operation is performed only on the nonzero UE. If the K-means method divides UEs into κ groups, we set κ masks for the groups. For example, a chromosome [4, 13, 12, 1, 7, 1, 9, 13, 4, 13] contains 10 UEs as depicted in Fig. 3. If the number of K-means groups is 3, the UE groups are:

$$[p_2 = 13,\ p_8 = 13,\ p_{10} = 13],$$
$$[p_3 = 12,\ p_7 = 9],$$
$$[p_1 = 4,\ p_4 = 1,\ p_5 = 7,\ p_6 = 1,\ p_9 = 4]$$

where the MEC servers 9 and 12 are closer. Servers 1, 4 and 7 are also not far apart. The three corresponding group masks are:

$$\mathbf{g}_1 : [0, 1, 0, 0, 0, 0, 0, 1, 0, 1],$$
$$\mathbf{g}_2 : [0, 0, 1, 0, 0, 0, 1, 0, 0, 0],$$
$$\mathbf{g}_3 : [1, 0, 0, 1, 1, 1, 0, 0, 1, 0].$$

If we add the three mask vectors vertically, we get an all-ones vector [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]. The auxiliary functions are initialized according to the group of masks,

$$\begin{aligned} &\min f_k(\mathbf{x}) = \ell(\mathbf{x}.*\mathbf{g}_k), \\ &s.t. \quad \mathbf{x} \in \Omega, k \in [1, \kappa] \end{aligned} \tag{7}$$

where $f_k(\mathbf{x})$ is the auxiliary task fitness function, $\mathbf{g}_k$ is the mask of group k, κ is the number of K-means groups, and $.*$ means element-wise multiplying of vector. f_k shares the same form of $f_0(\mathbf{x})$, but the computation is constrained to the UEs within a single group. The number of auxiliary tasks is determined by the number of groups. Poor design of an auxiliary task can lead to negative knowledge transfer and affect the efficiency of the main task. The motivation of UE grouping and constructing auxiliary tasks based on UE groups is to facilitate effective positive knowledge transfer and prevent negative knowledge transfer. Each UE exhibits obvious geographical attributes, which guides the selection of suitable MEC servers including the nearest server and closer servers with ample computational resources. A group of UEs within the same geographical range has a set of suitable MEC servers that are close in proximity. By restricting evolutionary operators to the suitable servers for a group of UEs with

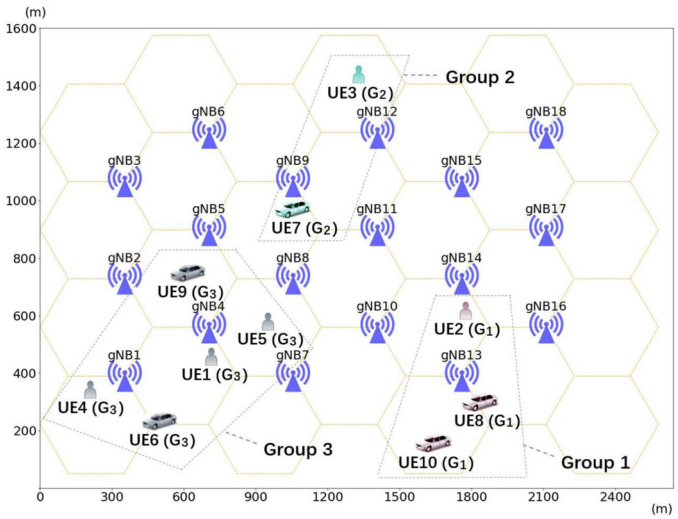


Fig. 3 The clustered groups of 10 UE. UE [2, 8, 10] are in group 1, UE [3, 7] are in group 2, and UE [1, 4, 5, 6, 9] are in group 3

strong geographical correlation, positive knowledge transfer more likely occurs. Furthermore, this approach can avoid the selection of distant servers, thereby mitigating the risk of negative knowledge transfer. Another benefit of adaptive auxiliary task design is that we use the homogeneous fitness evaluation function by adding a group mask based on physical location for each auxiliary task, as such no space transformation or mapping is required between the tasks and knowledge transfer could be more efficient.

Note that the proposed algorithm operates once within each time slot. Given a time slot, the initialization is performed at the beginning and K-means clustering is conducted to group the UEs and calculates the optimal migration strategy for the current time slot. For the next time slot when the UE positions change, the algorithm is re-run from scratch to obtain a new optimal migration strategy. The goal of MFMA-AAT is to optimize user profile migration within a single time slot. Currently, MFMA-AAT cannot address dynamic changes directly. Nevertheless, there could be a compromise solution by treating the dynamic behavior of users across different time slots as a serial of 'static' optimization problems. MFMA-AAT could be run multiple times to solve the problems sequentially. If the length of a time slot is set to small enough, the compromise solution could also accommodate agile service migration for rapid fluctuations in user demand.

In addition to the fitness evaluations of main task, defined in Eq. (6), and auxiliary tasks, defined in Eq. (7), the following properties should defined to enable EMT:

1. *Factorial cost*: The factorial cost of a solution $\mathbf{x}$ with respect to a task i is defined as $f_i(\mathbf{x})$.
2. *Factorial rank* The factorial rank of a solution $\mathbf{x}$ on task i indicates the rank of $\mathbf{x}$ in the population that is sorted in ascending order with respect to $f_i(\cdot)$.
3. *Skill factor* The skill factor of a solution $\mathbf{x}$ is the index of the task on which $\mathbf{x}$ is ranked the highest.
4. *Scalar fitness* The scalar fitness of a solution $\mathbf{x}$ is defined as the reciprocal of the factorial cost of $\mathbf{x}$ on skill factor task.

4.2 Adaptive auxiliary task selection

The multifactorial evolutionary process of MFMA-AAT is similar to the classical MFEAs, where the first task in MFEA is the main task and the second task in MFEA is an active auxiliary task. In each generation, we select an auxiliary task among κ auxiliary tasks as the active auxiliary task. Only the active auxiliary participates in the evaluation to prevent too much computation. The selection is implemented with roulette wheel selection. In the first generation, each auxiliary task is equally assigned a selection probability $\{\epsilon_k{=}1/\kappa|\epsilon_1, \epsilon_2, ..., \epsilon_k, ..., \epsilon_\kappa\}$. After an evolution generation n, MFMA-AAT records the optimal fitness value of the main task with auxiliary task k being selected as the active task, namely, $f_0^k(\mathbf{x}^{n\star})$, where $\mathbf{x}^{n\star}$ is the individual with the lowest fitness value in terms of the main task in the n-the generation. Based on $f_0^k(\mathbf{x}^{n\star})$, we can calculate the drop rate δ of the active auxiliary task in the n-th generation as follows:

$$\delta_k(n) = \frac{|f_0^k(\mathbf{x}^{n\star}) - f_0^k(\mathbf{x}^{(n-1)\star})|}{f_0^k(\mathbf{x}^{(n-1)\star})} \tag{8}$$

where $f_0^k(\mathbf{x}^{(n-1)\star})$ indicates the lowest fitness value in the previous generation. δ_k quantifies the improvement of the main task when using task k as the active auxiliary task. The selection probability ϵ_k is accordingly increased by the percentage of δ_k, i.e., $\epsilon_k = \epsilon_k(1 + \delta_k)$, while the selection probabilities of other inactivated auxiliary tasks are decreased on average accordingly, i.e., $\epsilon_i = \epsilon_i - \epsilon_k\delta_k/(\kappa-1)$ $\forall i \neq k$, to keep $\sum_1^\kappa \epsilon_k = 1$. Note that the active auxiliary task is selected dynamically based on probabilities, each auxiliary task has a chance to be activated and its effectiveness can then be evaluated. An auxiliary task contributing more to the overall convergence of the main task is more likely selected as active auxiliary task.

4.3 Evolutionary operators

In our multifactorial evolutionary process, we use a grouped uniform crossover operator to randomly exchange selected genes if they both belong a same group between two parent chromosomes. The used mutation operator is a swap mutation that randomly selects two genes on a chromosome and interchanges their values, promoting diversity in the population and avoiding local optimum. Elitist selection strategy is employed for both parent and offspring populations.

4.3.1 Grouped uniform crossover

Given a pair of parent chromosomes $\mathbf{x}^a$ and $\mathbf{x}^b$ in the population,

$$\mathbf{x}^a : [\mathbf{c}_1^a, \mathbf{c}_2^a, \mathbf{c}_3^a, ..., \mathbf{c}_{i-1}^a, \mathbf{c}_i^a, ..., \mathbf{c}_\eta^a],$$
$$\mathbf{x}^b : [\mathbf{c}_1^b, \mathbf{c}_2^b, \mathbf{c}_3^b, ..., \mathbf{c}_{i-1}^b, \mathbf{c}_i^b, ..., \mathbf{c}_\eta^b],$$

the grouped uniform crossover operation loops over each position i in the chromosome, if $\mathbf{c}_i^a$ and $\mathbf{c}_i^b$ belong a same group, then generates a random probability to judge whether $\mathbf{c}_i^a$ and $\mathbf{c}_i^b$ are to be exchanged. Compared with single-point or two-point crossover, the grouped uniform crossover has more fine-grained exchange with positive knowledge. The new pair of chromosomes is described as follows:

$$\mathbf{x}^{a'} : [\mathbf{c}_1^a, \mathbf{c}_2^b, \mathbf{c}_3^a, ..., \mathbf{c}_{i-1}^b, \mathbf{c}_i^a, ..., \mathbf{c}_\eta^b]$$
$$\mathbf{x}^{b'} : [\mathbf{c}_1^b, \mathbf{c}_2^a, \mathbf{c}_3^b, ..., \mathbf{c}_{i-1}^a, \mathbf{c}_i^b, ..., \mathbf{c}_\eta^a].$$

Algorithm 2 Memetic operator

Input: Population X, population size λ, number of UE η, number of MEC servers ω, local search probability p^l
Output: Optimized population X′

```
X′ ← X
for z = 1 to λ*p^l do
  for i = 1 to η do
    for j = 1 to ω do
      Identify ξ* according to Eq. (10)
      sim[j] = Similarity(ξ_ij, ξ*) (Eq. 11)
      if sim[j] > sim_threshold then
        φ_i.add(j)
      end if
    end for
    for ψ in φ_i do
      X_z[i] ← ψ
      val = f_0(X_z)
      if val < optimal_fitness then
        optimal_fitness ← val
        X′_z ← X_z
      end if
    end for
  end for
end for
```

4.3.2 Swap mutation

Given a chromosome, the swap mutation operation selects two random gene positions and then swaps them. For example, if the select mutation positions are i and j in a chromosome $\mathbf{x}$, i.e.,

$$\mathbf{x}^a : [\mathbf{c}_1^a, \mathbf{c}_2^a, \mathbf{c}_i^a, ..., \mathbf{c}_j^a, ..., \mathbf{c}_\eta^a],$$

the values of $\mathbf{x}_i^a$ and $\mathbf{x}_j^a$ are swapped:

$$\mathbf{x}^{a'} : [\mathbf{c}_1^a, \mathbf{c}_2^a, \mathbf{c}_j^a, ..., \mathbf{c}_i^a, ..., \mathbf{c}_\eta^a].$$

4.3.3 Elitist selection

The selection operator is based on the elitism strategy with linear rank selection. The chromosomes from both the parent and offspring populations are sorted according to their fitness values, and the best chromosome with the lowest user-perceived latency is first maintained. Then, the top-ranked chromosomes have a higher probability of selection without repetition.

4.4 Memetic operator

MFMA-AAT proposes a community detection method that combines the physical features of MEC servers to reduce the combinatorial space, and subsequently searches in the neighborhood using hill climbing [48]. The outline of memetic search is summarized in Algorithm 2.

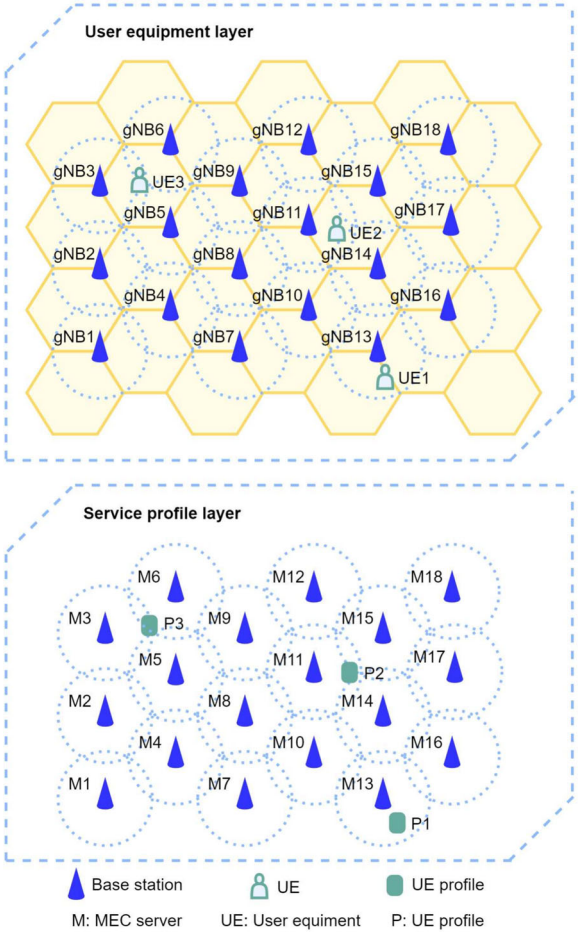


Fig. 4 MEC network layers. In an MEC network, the UE layer is physical, and the service profile layer is virtual. Each UE has a service profile only on an MEC server

4.4.1 Community detection

in an MEC network, the placement of a UE profile on an edge server depends on the UE location, server computing resources, and various other factors. Figure 4 shows an MEC network divided into a virtual service profile layer and a physical UE layer. The profile is often placed on the closest edge server to the user. However, the real-world multi-application environment is complex, and various factors like UE location, server congestion, computing capacity, user priority, and type of edge server influence the user-perceived latency. To address this, we propose a community detection method based on cosine similarity to identify suitable MEC servers for profile placement based on three critical attributes. The community detection method compares critical attributes of every server with an ideal placement. The MEC servers similar to the ideal server form a community $\boldsymbol{\phi}_i$ for u_i, serving as a basis for local search optimization. The three critical attributes of u_i in an MEC server j are the physical location

ξ_{ij}^1, server congestion ξ_{ij}^2, and computing capacity ξ_{ij}^3:

$$\begin{aligned}\xi_{ij}^1 &= \text{Hop}(u_i, j),\\ \xi_{ij}^2 &= \sum r_j + q_j,\\ \xi_{ij}^3 &= \tau_j,\end{aligned} \tag{9}$$

where $\text{Hop}(u_i, j)$ is the hop distance from the MEC server that belongs to the UE-located base station to the profile-placed MEC server. $r_j + q_j$ is the computation requirements and queues of all UEs at server j, and τ_j is the computation capability of server j. We use a vector $\boldsymbol{\xi}_{ij} = (\xi_{ij}^1, \xi_{ij}^2, \xi_{ij}^3)$ to denote the critical attributes for profile at MEC server j. The ideal placement of u_i achieves the optimal performance for each attribute. The values of optimal vector $\boldsymbol{\xi}^\star = (\xi^{*1}, \xi^{*2}, \xi^{*3})$ are the lowest hop distance, the lowest congestion, and the highest computing capacity that a profile can obtain in the MEC system.

$$\begin{aligned}\xi^{*1} &= \min_{j\in[1,\omega]}(\xi_{ij}^1),\\ \xi^{*2} &= \min_{j\in[1,\omega]}(\xi_{ij}^2),\\ \xi^{*3} &= \max_{j\in[1,\omega]}(\xi_{ij}^3),\end{aligned} \tag{10}$$

However, The MEC system is hard to provide ideal resources. Therefore, we tend to seek a candidate MEC server with high similarity to the ideal placement $\boldsymbol{\xi}^\star$. The computation of cosine similarity between $\boldsymbol{\xi}_{ij}$ and $\boldsymbol{\xi}^\star$ is given by:

$$\text{Similarity}(\boldsymbol{\xi}_{ij}, \boldsymbol{\xi}^\star) = \frac{\sum_{\iota=1}^{3}\left(\xi_{ij}^{\iota} * \xi^{\star\iota}\right)}{\sqrt{\sum_{\iota=1}^{3}(\xi_{ij}^{\iota})^2} * \sqrt{\sum_{\iota=1}^{3}(\xi^{\star\iota})^2}} \tag{11}$$

The samples of community detection are illustrated in Fig. 5. We calculate the cosine similarity of attribute vectors between each server and the ideal server, and a higher value indicates a higher degree of similarity. We perform rank-based selection on the cosine similarity results for each profile to every server, which allows us to select a set of servers as the search neighborhood. We empirically investigated the similarity threshold between 0.85 and 0.95 and identified 0.9 as the best setting.

4.4.2 Local search

Local search leverages hill climbing and community detection to enhance the search efficiency. As depicted in Fig. 6, during a time slot, user-specific profiles are placed in suitable MEC servers by detecting communities based on similarity comparisons of attributes. In other words, the search scope for profile placement per user is narrowed down from all MEC servers to the community set, which can overlap without causing conflicts. The candidate MEC servers in a community are ranked by similarity to facilitate the hill climbing process.

In the memetic method, the $\lfloor \eta * \text{p}^1 \rfloor$ top-ranked chromosomes are selected for local search. The neighborhood search space is generated with the detected communities based on each chromosome. Position i is randomly selected on the chromosome, and the gene is replaced with another MEC server in the corresponding community. If the number of candidate servers in a community cannot reach the neighborhood size, the next gene is considered for replacement. For an individual

$$[10, 11, 1, ..., \eta],$$

if the initial selected position i is 1, then the neighborhood is given by:

$$[\mathbf{13}, 11, 1, ..., \eta]$$

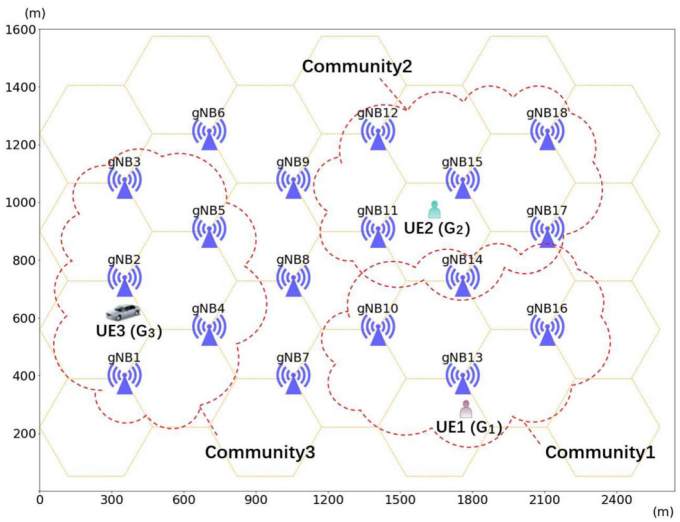


Fig. 5 Community detection. A UE profile can be placed on a subgroup of MEC servers representing a community that can provide sufficient computing capacity

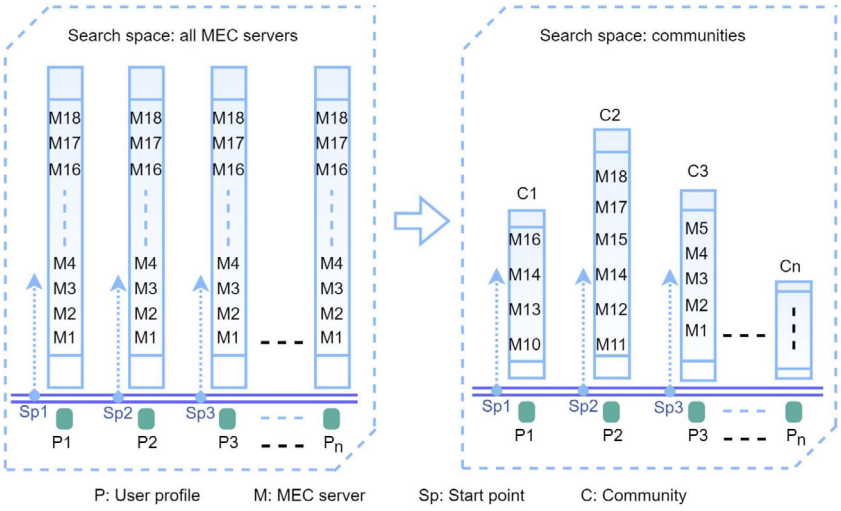


Fig. 6 The memetic method. Each randomly generated start point is used to establish a search within its corresponding community

[**14**, 11, 1, ..., η]

[**16**, 11, 1, ..., η]

[10, **12**, 1, ..., η]

[10, **14**, 1, ..., η]

[10, **15**, 1, ..., η]

...

The neighborhood size is usually set as $\lfloor 1/p^l \rfloor$. If the next chromosome in the neighborhood has an optimal fitness value, the climbing search continues until it encounters a worse value.

4.5 Termination condition

In this study, we use the maximum number of fitness evaluations (Max^E) as the termination criterion. The conventional EAs have a fixed number of fitness function evaluations per generation, whereas memetic algorithms vary in the number of fitness function evaluations as hill climbing performs local search without a fixed count. Moreover, memetic operator and local search could have other extra computational cost rather than fitness evaluation, Nevertheless, in this study, such costs are illegible compared to that of the fitness evaluation. Hence, Max^E can serves as a reasonable computational budget to ensure fair comparison of the algorithms.

4.6 Time complexity

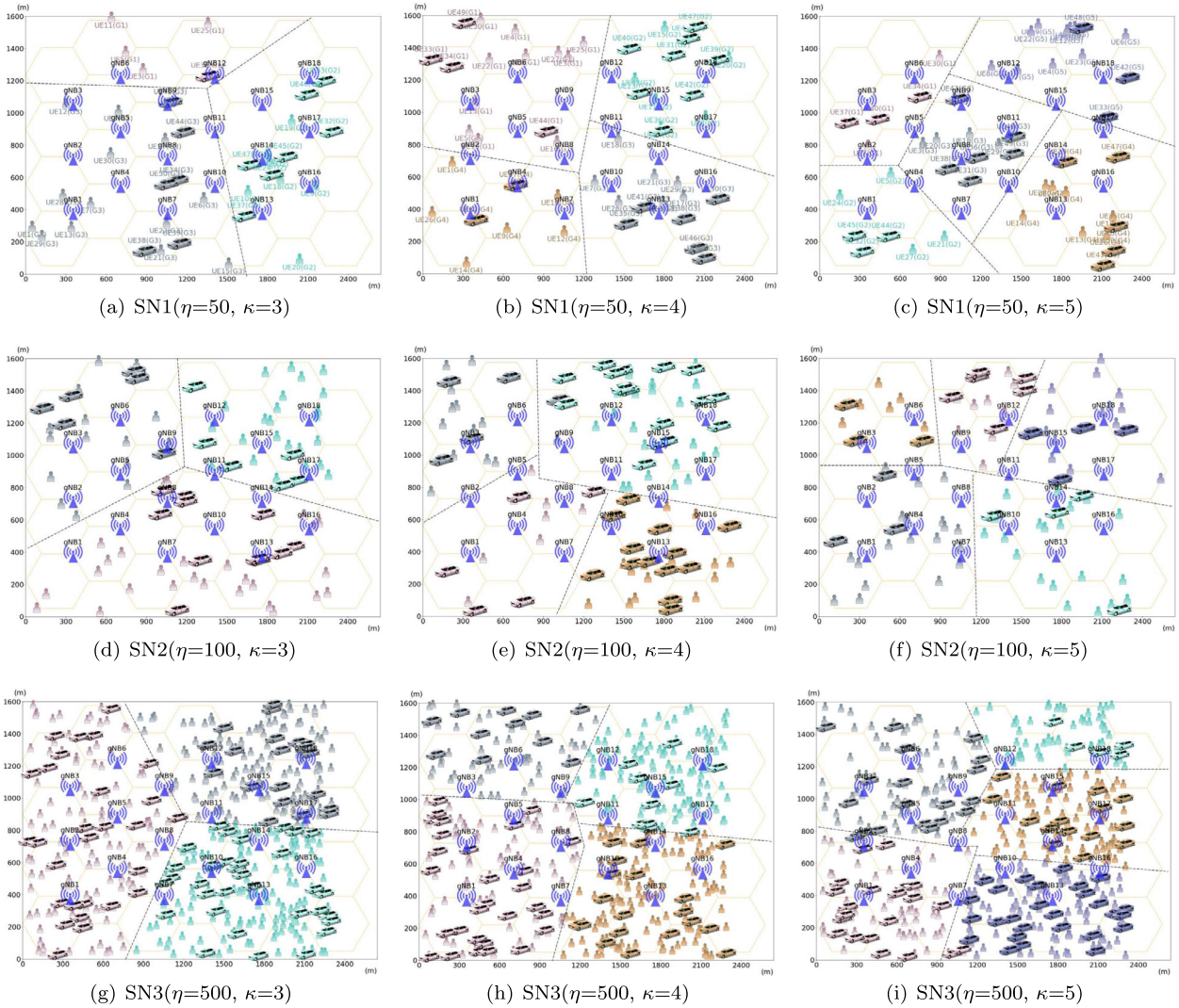
In computational time cost, MFMA-AAT boasts K-means clustering and memetic search over MFEAs. The time complexity of the K-means clustering algorithm is $O(\eta\kappa\nu)$, where η is the number of UEs, κ is the number of clusters (typically in the range of 3–5), and ν is the number of iterations in the clustering process. In practice, with $\eta = 500$, the average value of ν is 20 times, so both κ and ν can be considered as constants, thus yielding a time complexity of $O(\eta)$. Additionally, K-means clustering is only executed once during chromosome initialization, taking only a few hundred milliseconds and having a minimal impact on overall time. The time complexity of memetic search, based on community structure, is $O(\eta\omega Run^G)$, where η is the number of UEs and ω is the number of servers. It involves computing the similarity for each server and selecting a group of suitable servers to reduce the search space. Given that $\omega = 18$ was a constant in our experiments, the actual time complexity of the memetic algorithm is $O(\eta Run^G)$. The time complexity of the multifactorial evolution is linearly correlated with the number of fitness evaluations, $O(Max^E)$, with either the main task or the auxiliary task counting as a fitness calculation. Max^E is generally set to $\lambda * Run^G$, where λ is the population size defined as a constant, so the time complexity of MFMA-AAT is $O(\eta Max^E)$.

5 Experimental studies

5.1 Test MEC networks

To study the performance of MEC systems, we use a general MEC network model consisting of ω base stations and η randomly positioned UEs. The network was generated using the wireless network simulation platform [49] that allows for customization of the base stations and UEs, as shown in Fig. 7. In our experiment, the MEC test network contains $\omega = 18$ servers, as depicted in Fig. 1.

The generation of the MEC test network involves two steps. The first step is setting MEC server data. We arrange the positions of the base stations to form an MEC network in the physical space. The MEC server topology network is



(a) SN1(η=50, κ=3) (b) SN1(η=50, κ=4) (c) SN1(η=50, κ=5)

(d) SN2(η=100, κ=3) (e) SN2(η=100, κ=4) (f) SN2(η=100, κ=5)

(g) SN3(η=500, κ=3) (h) SN3(η=500, κ=4) (i) SN3(η=500, κ=5)

Fig. 7 Clustering results of UEs with $\kappa = \{3, 4, 5\}$ in different MEC networks. The configurations of networks SN1-3 are summarized in Table 3. η is the number of UEs and κ is the number of K-means groups

then generated by pairing each MEC server with a base station. Afterward, we calculate the hop count table between each base station and set the computation capability for each MEC server. The second step is generating UE data. We give random initial positions, types, and moving speeds for each UE. Additionally, we assign attributes such as the requested computational workload in each time slot. The network we generate have a fixed number of MEC servers with predetermined computation capabilities. However, the attributes such as the number of UEs, types, and other characteristics can vary to create different types of test networks.

Both the number of UEs and congestion level of the servers in the MEC system could affect the average user-perceived latency, so we considered several simulated network scenarios. In Table 3, the congestion level denotes the level of crowdedness of an MEC server in a time slot. We experiment with three different UE counts of 50, 100 and 500. For congestion level, we also have three levels from $C1$ to $C3$, and the congestion level is getting heavier. Hence, $SN1$ represents a network with few UEs and light congestion, $SN2$ is a network with normal traffic flow of pedestrians and vehicles, and $SN3$ is a highly congested network with many UEs assigned to certain MEC servers. The experimental hardware platform is equipped with an $i7-1255U$ ($1.7Ghz$) processor and $16GB$ of RAM.

5.2 Parameter settings

To investigate the effects of the evolutionary operators, we compared different crossover and mutation strategies. Particularly, we considered single-point, two-point and grouped uniform crossover. For mutation operator, we taken into

Table 3 Parameters of MEC network

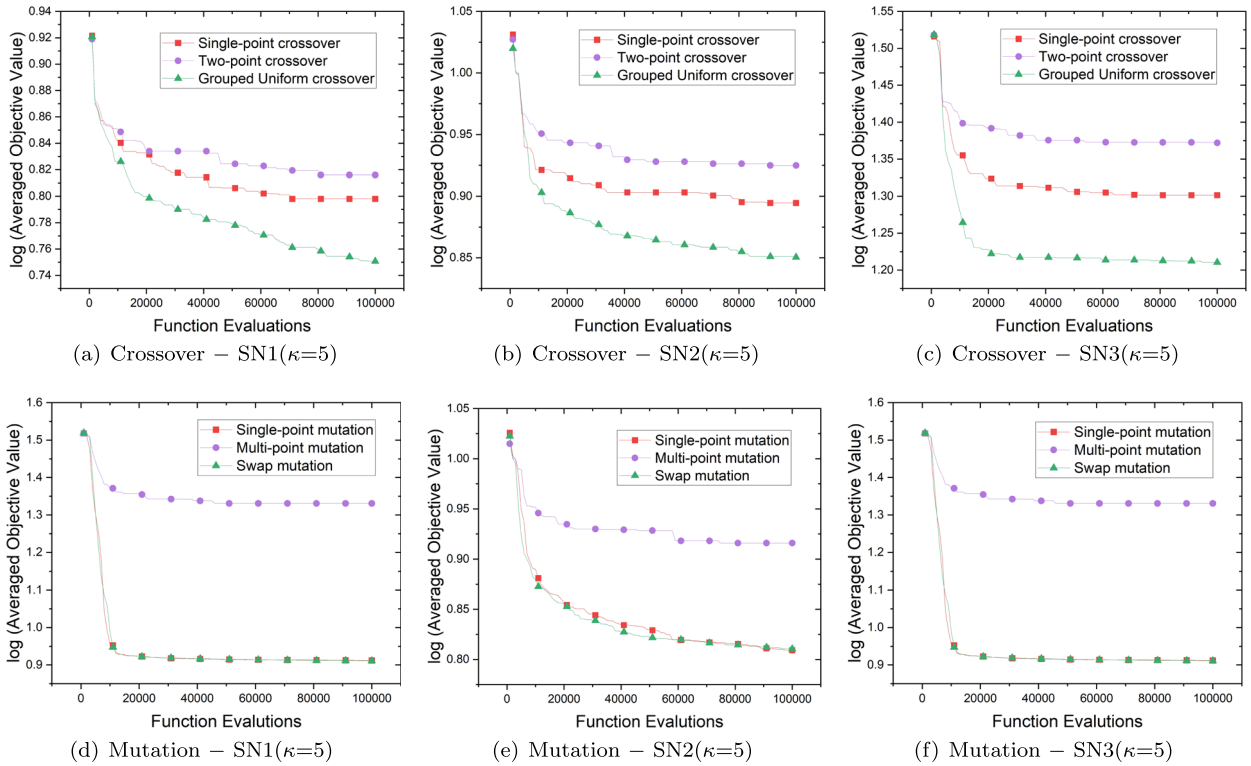
Network	UE size	Congestion level	Metric
SN1	50	C1	Latency
SN2	100	C2	Latency
SN3	500	C3	Latency

account the single-point, multi-point, and swap mutations. The number of tasks in MFMA-AAT is determined by the number of clustered groups. Typically, a reasonable approach is to align the number of groups with the count of geographical regions. This grouping strategy is consistent across different UE counts and congestion levels, as UEs predominantly move within a single region. Within our dataset, a more appropriate choice for κ is within the range of 3–5.

The average convergence traces of MFMA-AAT over 20 runs using different crossover strategies on the networks SN1-SN3 with representative cluster number $\kappa = 5$ are shown in Fig. 8a–c. Grouped uniform crossover is shown to perform better than the other two strategies. The results of MFMA-AAT using different mutation operators are shown in Fig. 8d–f, where single-point mutation and swap mutation outperform multi-point mutation, and swap mutation is slightly better than single-point mutation.

Since MFMA-AAT tends to search within the same geographic area, grouped uniform crossover can complement more to the algorithm with better exploration capability in the evolutionary search based on geographic clustering. Swap and single-point mutations have relatively small changes to chromosomes, so they are more conductive to the stable evolution of the best individuals in the population, and swap mutation can control the harmful mutation to a lower range. Considering the overall performance, we adopted grouped uniform crossover and swap mutation in MFMA-AAT.

To optimize the performance MFMA-AAT, we also fine-tuned its critical parameters, including the crossover probability p^c, the mutations probability p^m, and the memetic search probability p^l determines the probability of a chromosome undergoing memetic search. Therefore, we conducted trial-and-error evaluations of each parameter to determine the optimal settings for MFMA-AAT in the MEC network experiments. To ensure that convergence is observable, we have established a consistent computational budget of Max^E=100,000. To tune p^c, we first fixed other key parameters as (p^m=0.0, p^l=0.0, λ=100, Max^E=100,000) and then tested the values of p^c from 0.1 to 0.9 increasing by 0.1. The comparison results of using different p^c values are reported in Table 4. The significance of performance difference by using different values is tested with Friedman test and the significance of performance difference between



(a) Crossover − SN1(κ=5) (b) Crossover − SN2(κ=5) (c) Crossover − SN3(κ=5)

(d) Mutation − SN1(κ=5) (e) Mutation − SN2(κ=5) (f) Mutation − SN3(κ=5)

Fig. 8 Average convergence traces using different crossover and mutation operators on SN1-3 networks ($K = 5$) over 20 runs

Table 4 Parameters comparison of crossover over 20 independent trials

Data	p^c=0.1	p^c=0.2	p^c=0.3	p^c=0.4	p^c=0.5	p^c=0.6	p^c=0.7	p^c=0.8	p^c=0.9	Friedman *P*-value
SN1(κ=3)	6.07±0.12(+)	6.09±0.04(+)	5.87±0.06(+)	5.95±0.09(+)	5.95±0.14(+)	6.1±0.07(+)	5.88±0.1(+)	**5.74±0.11**	5.77±0.1(+)	3.00E-02
SN1(κ=4)	6.1±0.04(+)	5.87±0.12(+)	5.93±0.09(+)	5.89±0.09(+)	5.92±0.08(+)	5.98±0.04(+)	5.75±0.07(+)	**5.58±0.09**	5.66±0.03(+)	2.00E-02
SN1(κ=5)	6.08±0.05(+)	6.05±0.04(+)	6.17±0.0047(+)	5.96±0.06(+)	5.92±0.02(+)	6.01±0.02(+)	5.87±0.07(+)	5.76±0.09(+)	**5.7±0.07**	5.20E-03
SN2(κ=3)	8.0±0.01(+)	8.02±0.01(+)	7.94±0.08(+)	8.0±0.02(+)	7.83±0.08(+)	7.86±0.09(+)	7.87±0.04(+)	**7.7±0.09**	7.82±0.05(+)	1.00E-02
SN2(κ=4)	7.89±0.04(+)	7.88±0.06(+)	7.8±0.07(+)	7.81±0.05(+)	7.74±0.06(+)	7.79±0.05(+)	7.76±0.04(+)	7.62±0.1(+)	**7.61±0.04**	2.00E-02
SN2(κ=5)	7.46±0.09(+)	7.48±0.1(+)	7.45±0.07(+)	7.5±0.04(+)	7.45±0.06(+)	7.43±0.07(+)	7.35±0.04(+)	**7.29±0.01**	7.29±0.02(+)	5.00E-02
SN3(κ=3)	14.44±0.08(+)	14.16±0.04(+)	13.72±0.18(+)	13.67±0.13(+)	13.44±0.32(+)	13.29±0.23(+)	12.78±0.61(+)	**12.33±0.3**	12.91±0.04(+)	6.20E-03
SN3(κ=4)	14.56±0.08(+)	14.37±0.03(+)	14.12±0.07(+)	13.79±0.15(+)	13.79±0.07(+)	13.81±0.08(+)	13.66±0.03(+)	**13.18±0.17**	13.27±0.07(+)	3.80E-03
SN3(κ=5)	18.15±0.08(+)	18.12±0.07(+)	17.95±0.14(+)	17.84±0.06(+)	17.48±0.1(+)	17.65±0.06(+)	17.56±0.08(+)	**16.93±0.17**	17.15±0.17(+)	4.20E-03
Best	0	0	0	0	0	0	0	**7**	2	

Data	p^c=0.1	p^c=0.2	p^c=0.3	p^c=0.4	p^c=0.5	p^c=0.6	p^c=0.7	p^c=0.8	p^c=0.9	
SN1(κ=3)	4.00E-02	1.00E-02	2.30E-01	1.20E-01	1.70E-01	2.00E-02	2.50E-01	–	8.40E-01	T-test *P*-value
SN1(κ=4)	1.70E-03	5.00E-02	2.00E-02	3.00E-02	2.00E-02	4.70E-03	1.10E-01	–	3.00E-01	T-test *P*-value
SN1(κ=5)	3.70E-03	3.50E-03	7.00E-04	2.00E-02	1.00E-02	3.70E-03	7.00E-02	5.10E-01	–	T-test *P*-value
SN2(κ=3)	1.00E-02	9.00E-03	5.00E-02	1.00E-02	2.30E-01	1.60E-01	8.00E-02	–	2.00E-01	T-test *P*-value
SN2(κ=4)	2.10E-03	7.60E-03	3.00E-02	1.00E-02	7.00E-02	2.00E-02	3.00E-02	9.70E-01	–	T-test *P*-value
SN2(κ=5)	6.00E-02	5.00E-02	3.00E-02	1.60E-03	2.00E-02	4.00E-02	7.00E-02	–	7.00E-01	T-test *P*-value
SN3(κ=3)	6.00E-04	1.00E-03	4.90E-03	4.20E-03	2.00E-02	2.00E-02	4.00E-01	–	5.00E-02	T-test *P*-value
SN3(κ=4)	5.00E-04	7.00E-04	2.10E-03	2.00E-02	1.00E-02	1.00E-02	2.00E-02	–	5.50E-01	T-test *P*-value
SN3(κ=5)	7.00E-04	8.00E-04	2.50E-03	1.90E-03	2.00E-02	4.50E-03	8.70E-03	–	2.60E-01	T-test *P*-value

The best performance values are shown in bold.

the best setting and each other settings is tested with t-test. The corresponding test p-values are also reported in Table 4. It can be seen that with $p^c \geq 0.6$, MFMA-AAT obtains better performance and the optimal setting is $p^c = 0.8$. We conducted similar tuning experiments on other parameters. For p^m, the test results are listed in Table 5 with (p^c=0.8, p^l=0.0, λ=100, Max^E=100,000), and p^m is selected from a smaller range, i.e., from 0.1 to 0.4. At mutation probability p^m of 0.1, MFMA-AAT achieved the best performance. The memetic operator performs a local search for a chromosome in its neighborhood. Probability p^l is used to control the proportion of chromosomes for memetic search. We tested p^l from 0.1 to 0.9 and fixed other parameters (p^c=0.0, p^m=0.0, λ=100, Max^E=100,000) to suppress their interference. The test results are listed in Table 6. The optimal setting of p^l for MFMA-AAT is obtained at 0.5.

The above tests all run over 20 independent trials then we obtained the optimal parameter settings listed in Table 7.

5.3 Comparison results

To evaluate the proposed MFMA-AAT, we compared it with traditional MEC algorithm GT [12], conventional genetic algorithm (GA) [50], and the state-of-the-art MFEAs including MFEA-II [19], MFEA-AKT* [34], and MFEA-BLKT* [40], MTCMO* [23]. We also involved MFEA-II (i.e., MFEA algorithm without local search and AAT), MFEA-AAT (i.e., MFMA-AAT without local search), MFMA (i.e., MFMA-AAT without AAT), and MFMA-RAT (i.e., MFMA-AAT utilizes random groups instead of clustered groups) in ablation study to evaluate the effects of the key components of MFMA-AAT.

In MFEA-II, two optimization tasks, i.e., ℓ defined in Eq. (5) and l^α defined in Eq. (1) were considered. Here l^α is empirically set as the auxiliary task as it is the most similar subtask to the main task ℓ. MFEA-AKT* is modified from the original MFEA-AKT [34] that dynamically selects different crossover operators to control the efficiency of knowledge transfer in each generation. Instead of selecting crossover, in MFEA-AKT* we applied the same selection strategy to select auxiliary tasks from $l^\alpha(t)$ (Eq. 1), $l^\beta(t)$ (Eq. 2), and $l^\gamma(t)$ (Eq. 3).

MFEA-BLKT* is modified from the original algorithm proposed in [40] to make it suitable to solve the target problem in this study. MTCMO* is based on MFMA and incorporates a dynamic auxiliary task update strategy as presented in [23]. The auxiliary tasks are designed to be diverse in the early stage and focus on high quality in the later stage. Note that since there are no MFEAs implementing similar auxiliary task selection mechanism in the literature, the comparison to MFEA-II, MFEA-AKT*, MFEA-BLKT* and MTCMO* with the current configurations might not be absolutely fair. Nevertheless, it could reflect the performance of the proposed MFMA-AAT to some extent.

When drawing the convergence trace in Fig. 9, 100 points are taken evenly in running time span, and each point on the horizontal axis denotes 0.25, 0.4 and 1.5 second for $SN1$, $SN2$ and $SN3$ data, respectively. Table 8 summarizes the average objective value comparison of all algorithms, and Fig. 9 shows the evolutionary process according to running time in three different networks with cluster number $\kappa = \{3, 4, 5\}$. The significance of performance difference among all algorithms is tested with Friedman test and the significance of performance difference between MFMA-AAT and each other algorithms is tested with t-test. The corresponding test p-values are also reported in Table 8. As the number of UEs and network load increased from $SN1$ to $SN3$, the user-perceived latency result also increased, yet MFMA-AAT always managed to achieve the significantly best result in the test networks. In SN3, the result in the initial phase drops rapidly while in the next phase is slower, and the differences in values between upper and lower algorithms are significant, which lead to what looks like early convergence, so the separate plots are made, as shown in Fig. 10.

In the case the amount of UE is small, i.e., $SN1$, the multitasking MFEAs outperform the single-task method GA, and the difference between the MFEAs is small. MFMA-AAT shows slight superiority to other methods. GT does not start from a random initial value like EAs, so the initial fitness value of GT is lower than EAs. GT converges very fast but mostly to local optima. Since GT is a deterministic method, it was not included in the significance test with other methods in Table 8. As the amount of UE increases, such as in $SN2$, the search space becomes larger and GT is more likely to get trapped into local optima. MFEAs can jump out of the local optima to detect a lower fitness value. All MFEAs outperform GT and GA thanks to the use of auxiliary tasks. MFEA-AKT* performs better than MFEA-II in SN1 and SN2 which suggests the dynamic selection of auxiliary tasks provides more positive knowledge transfer than using a constant auxiliary task. MFEA-BLKT* outperforms MFEA-AKT*, highlighting that employing K-means grouping to gene blocks during evolutionary operations promotes effective knowledge sharing and prevents negative knowledge transfer.

Compared with the MFEAs except MFMA-AAT, MFMA demonstrates superior outcomes by leveraging the physical location attribute to construct neighborhoods for local searches, thus hastening convergence. Particularly, MFEA-AAT outperforms MFMA, indicating AAT contributes more than local search. This is because AAT generates a set of auxiliary tasks based on the population's feature and selects the most efficient one for maximum positive knowledge transfer, based on the spatiotemporal information of the MEC network. Consequently, AAT achieves quicker convergence and detect more promising solutions for service migration.

Table 5 Parameters comparison of mutation over 20 independent trials

Data	p^m=0.1	p^m=0.2	p^m=0.3	p^m=0.4	Friedman *P*-value
SN1(κ=3)	**5.4±0.09**	5.95±0.08(+)	6.23±0.07(+)	6.43±0.09(+)	8.91E-29
SN1(κ=4)	**5.44±0.08**	5.93±0.08(+)	6.25±0.08(+)	6.44±0.11(+)	3.33E-28
SN1(κ=5)	**5.6±0.09**	6.08±0.1(+)	6.4±0.08(+)	6.57±0.11(+)	1.40E-27
SN2(κ=3)	**7.24±0.05**	7.59±0.07(+)	7.87±0.07(+)	8.11±0.07(+)	1.81E-30
SN2(κ=4)	**7.12±0.08**	7.49±0.07(+)	7.78±0.07(+)	8.05±0.08(+)	2.31E-30
SN2(κ=5)	**7.03±0.06**	7.42±0.09(+)	7.78±0.1(+)	8.13±0.1(+)	2.40E-30
SN3(κ=3)	**8.63±0.05**	9.9±0.14(+)	12.22±0.15(+)	15.07±0.2(+)	2.78E-32
SN3(κ=4)	**8.78±0.06**	10.63±0.16(+)	13.48±0.29(+)	16.77±0.31(+)	3.21E-32
SN3(κ=5)	**9.17±0.07**	12.03±0.19(+)	15.62±0.26(+)	19.03±0.32(+)	3.60E-32
count(*)	9	0	0	0	

Data	p^m=0.1	p^m=0.2	p^m=0.3	p^m=0.4	
SN1(κ=3)	–	5.77E-19	1.09E-26	8.64E-29	T-test *P*-value
SN1(κ=4)	–	1.09E-20	1.51E-28	2.27E-29	T-test *P*-value
SN1(κ=5)	–	1.30E-18	4.93E-28	5.31E-28	T-test *P*-value
SN2(κ=3)	–	1.20E-20	1.79E-28	1.51E-33	T-test *P*-value
SN2(κ=4)	–	1.30E-17	1.86E-26	1.74E-30	T-test *P*-value
SN2(κ=5)	–	1.24E-18	7.41E-28	1.17E-33	T-test *P*-value
SN3(κ=3)	–	7.41E-32	7.71E-48	1.08E-52	T-test *P*-value
SN3(κ=4)	–	1.47E-35	1.57E-41	1.87E-49	T-test *P*-value
SN3(κ=5)	–	4.82E-40	2.25E-48	4.10E-52	T-test *P*-value

The best performance values are shown in bold.

MTCMO* achieves better performance than MFMA and MFMA-RAT, illustrating the effectiveness of dynamically setting auxiliary tasks. Moreover, the greater the effective knowledge transfer from auxiliary tasks to the main task, the faster the convergence is facilitated.

MFMA-AAT outperforms MFEA-AAT and MFMA-RAT. The improvement of MFMA-AAT over MFEA-AAT is attributed to the addition of a memetic operator, which substantially improves the local convergence of the algorithm. The enhancement of MFMA-AAT over MFMA-RAT is evident in its utilization of clustered groups, which fosters more pronounced positive transfer effects than those achieved with random groups. Under high mobility at level $S3$ and heavy congestion at level $C3$ in $SN3$, the superiority of MFMA-AAT to other methods is more significant. With $\kappa = 4$ and 5 in $SN3$, it is evident that the more geographically relevant the UEs are, the more pronounced the benefits of MFMA-AAT become. Observations from $SN3$ in Fig. 9 show a rapid decline in the early stage, indicating MFMA-AAT's agility in service migration among denser UEs.

To sum up, the dynamic auxiliary task framework of MFMA-AAT is more aware at recognizing the movement patterns of UEs within the spatial domain, thus facilitating faster convergence in evolutionary computations.

5.4 Discussions

While traditional EMT algorithms rely on prior knowledge or constraint adjustment to manually define decomposed or similar auxiliary tasks, our approach MFMA-AAT leverages positional information within the physical dimension. This approach automatically determines auxiliary tasks in response to the positional information from the MEC system, thereby eliminating the need for manual assignment.

Theoretically, EMT necessitates that auxiliary tasks facilitate positive knowledge transfer to accelerate the convergence of the main task. MFMA-AAT constructs auxiliary tasks by clustering based on the physical locations of UEs, ensuring that UE profiles are generally migrated to MEC servers within the same region. This aligns with real-world scenarios where UEs, limited by spatial and temporal constraints, typically move within their clustered areas. This strategy guarantees that auxiliary tasks contribute positively to the knowledge transfer process. Reflecting on the experimental outcomes, we note that the AAT method incorporating clustering performs more effectively than its counterpart without this feature.

Concurrently, during the evolutionary computation and memetic search processes, the auxiliary tasks adopt the identical fitness evaluation as the main task. This alignment prevents the accuracy loss typically associated with

Table 6 Parameter comparison of local search over 20 independent trials

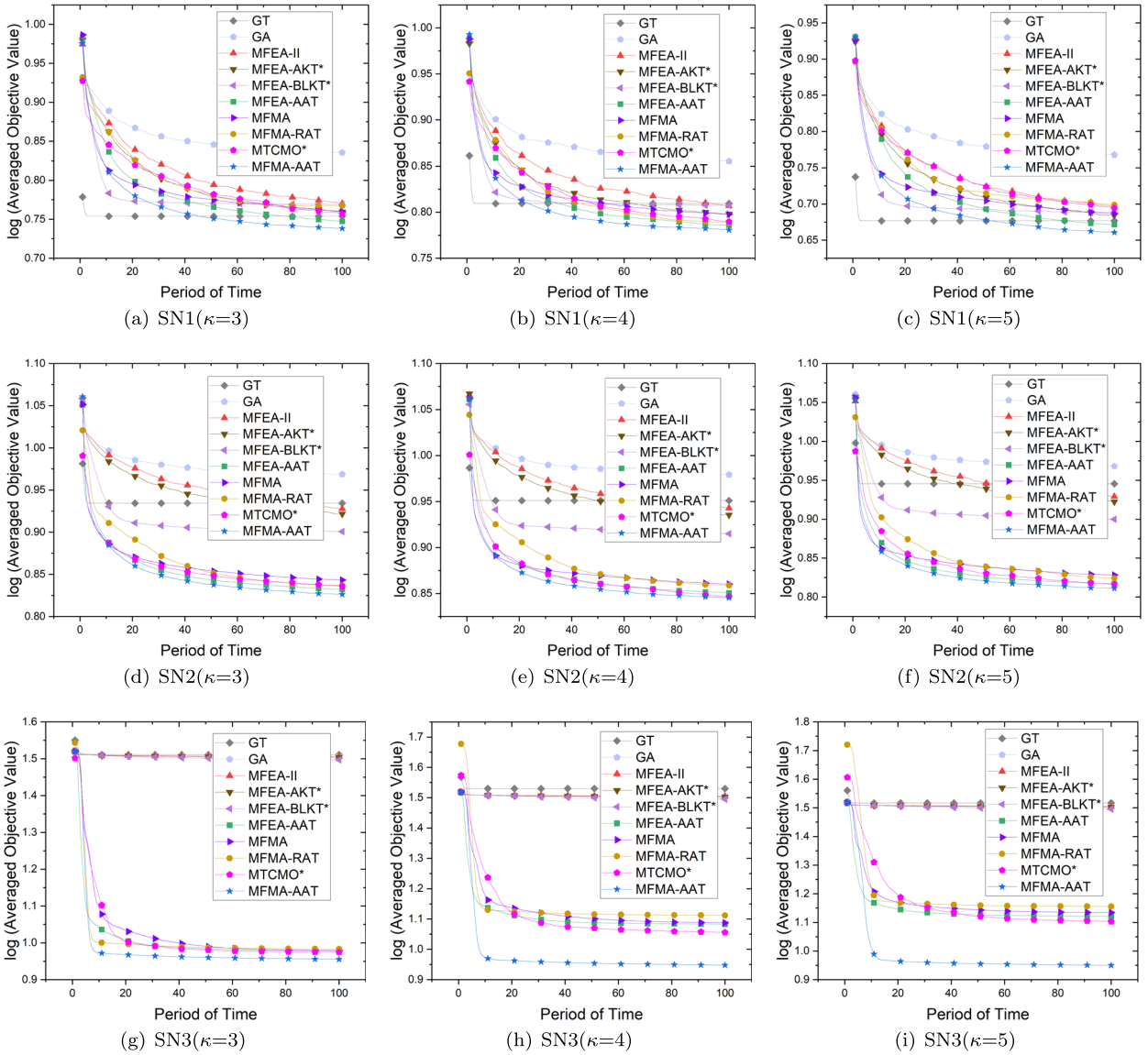
Data	p^l=0.1	p^l=0.2	p^l=0.3	p^l=0.4	p^l=0.5	p^l=0.6	p^l=0.7	p^l=0.8	p^l=0.9	Friedman *P*-value
SN1(κ=3)	5.48±0.1(+)	5.45±0.08(+)	5.5±0.08(+)	5.38±0.12(+)	**5.37±0.09**	5.5±0.08(+)	5.51±0.08(+)	5.46±0.08(+)	5.47±0.07(+)	5.11E-05
SN1(κ=4)	5.43±0.1(+)	5.43±0.11(+)	5.49±0.08(+)	5.37±0.09(+)	**5.36±0.09**	5.41±0.08(+)	5.42±0.08(+)	5.43±0.09(+)	5.44±0.12(+)	1.00E-02
SN1(κ=5)	5.6±0.11(+)	5.6±0.08(+)	5.57±0.09(+)	5.48±0.08(+)	**5.44±0.1**	5.59±0.11(+)	5.58±0.1(+)	5.58±0.07(+)	5.56±0.09(+)	1.20E-07
SN2(κ=3)	7.25±0.06(+)	7.24±0.05(+)	7.24±0.08(+)	7.24±0.06(+)	**7.23±0.05**	7.26±0.05(+)	7.24±0.07(+)	7.24±0.05(+)	7.24±0.05(+)	7.30E-01
SN2(κ=4)	7.13±0.06(+)	7.12±0.06(+)	7.11±0.08(+)	7.13±0.03(+)	7.11±0.06(+)	7.12±0.08(+)	7.11±0.05(+)	**7.07±0.05**	7.09±0.05(+)	3.00E-02
SN2(κ=5)	6.98±0.06(+)	7.01±0.07(+)	7.0±0.06(+)	6.98±0.06(+)	6.98±0.05(+)	7.01±0.07(+)	**6.97±0.08**	6.97±0.06(+)	7.01±0.08(+)	2.10E-01
SN3(κ=3)	8.61±0.04(+)	8.63±0.04(+)	8.61±0.03(+)	8.61±0.03(+)	**8.6±0.05**	8.61±0.04(+)	8.63±0.04(+)	8.61±0.04(+)	8.62±0.04(+)	5.10E-01
SN3(κ=4)	8.78±0.05(+)	8.76±0.04(+)	8.79±0.06(+)	**8.73±0.07***	8.77±0.06(+)	8.77±0.04(+)	8.77±0.04(+)	8.78±0.06(+)	8.77±0.05(+)	5.60E-01
SN3(κ=5)	9.21±0.09(+)	9.17±0.08(+)	9.2±0.08(+)	9.22±0.05(+)	**9.16±0.08**	9.21±0.06(+)	9.19±0.08(+)	9.21±0.07(+)	9.23±0.06(+)	1.00E-01
count(*)	0	0	0	1	**6**	0	1	1	0	

Data	p^l=0.1	p^l=0.2	p^l=0.3	p^l=0.4	p^l=0.5	p^l=0.6	p^l=0.7	p^l=0.8	p^l=0.9	
SN1(κ=3)	8.00E-04	1.00E-02	6.54E-05	9.50E-01	–	4.21E-05	1.65E-05	3.80E-03	4.00E-04	T-test *P*-value
SN1(κ=4)	5.00E-02	6.00E-02	6.62E-05	7.80E-01	–	1.30E-01	6.00E-02	3.00E-02	3.00E-02	T-test *P*-value
SN1(κ=5)	2.68E-05	5.16E-06	2.00E-04	1.70E-01	–	2.00E-04	1.00E-04	2.81E-05	5.00E-04	T-test *P*-value
SN2(κ=3)	3.30E-01	4.70E-01	6.30E-01	6.20E-01	–	1.30E-01	6.10E-01	8.30E-01	6.10E-01	T-test *P*-value
SN2(κ=4)	1.70E-03	8.90E-03	4.00E-02	7.16E-05	2.00E-02	1.00E-02	1.00E-02	–	2.10E-01	T-test *P*-value
SN2(κ=5)	6.30E-01	7.00E-02	1.90E-01	5.00E-01	4.80E-01	5.00E-02	–	8.60E-01	1.30E-01	T-test *P*-value
SN3(κ=3)	6.50E-01	1.50E-01	5.10E-01	6.00E-01	–	6.90E-01	7.00E-02	7.60E-01	2.80E-01	T-test *P*-value
SN3(κ=4)	1.00E-02	1.00E-01	1.00E-02	–	1.00E-01	3.00E-02	5.00E-02	4.00E-02	1.00E-01	T-test *P*-value
SN3(κ=5)	1.20E-01	9.20E-01	1.20E-01	9.30E-03	–	7.00E-02	2.60E-01	6.00E-02	1.00E-02	T-test *P*-value

The best performance values are shown in bold.

Table 7 Parameter settings of MFMA-AAT

Parameter	Description	Value
p^c	Probability of crossover	0.8
p^m	Probability of mutation	0.1
p^l	Probability of local search	0.5
λ	Size of the population	100
Run^G	Number of generations	1000
Max^E	Maximum number of fitness evaluations	100,000



(a) SN1(κ=3) (b) SN1(κ=4) (c) SN1(κ=5)

(d) SN2(κ=3) (e) SN2(κ=4) (f) SN2(κ=5)

(g) SN3(κ=3) (h) SN3(κ=4) (i) SN3(κ=5)

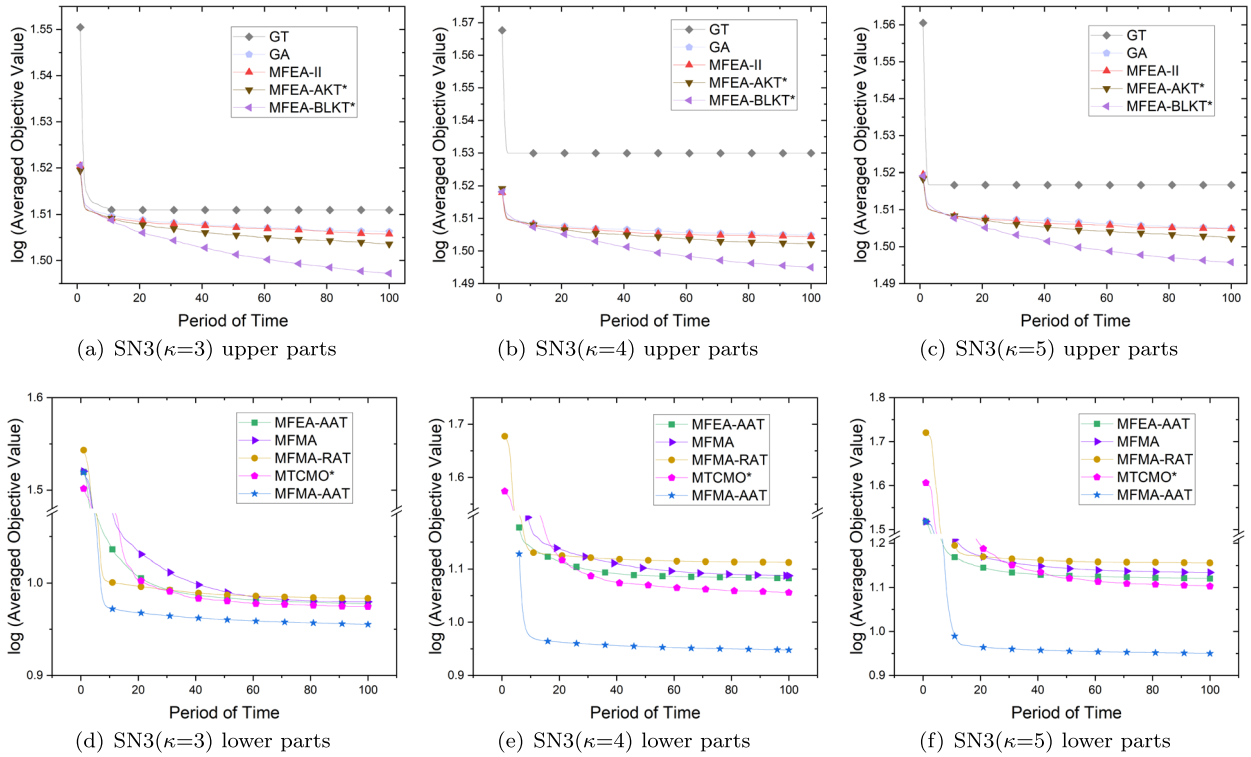
Fig. 9 Convergence trace of the compared algorithms on SN1-3 with different cluster numbers

Table 8 Average main task fitness values of the compared algorithms over 20 independent runs

Data	GT	GA	MFEA-II	MFEA-AKT*	MFEA-BLKT*	MFEA-AAT	MFMA	MFMA-RAT	MTCMO*	MFMA-AAT	Friedman *P*-value
SN1(κ=3)	5.67±0.02(+)	6.84±0.09(+)	5.89±0.06(+)	5.75±0.05(+)	5.87±0.09(+)	5.59±0.07(+)	5.75±0.09(+)	5.85±0.13(+)	5.71±0.1(+)	**5.47±0.05**	3.02E-26
SN1(κ=4)	6.45±0.06(+)	7.17±0.08(+)	6.41±0.06(+)	6.28±0.07(+)	6.42±0.11(+)	6.11±0.08(+)	6.28±0.06(+)	6.14±0.12(+)	6.16±0.07(+)	**6.04±0.08**	8.02E-30
SN1(κ=5)	4.75±0.05(+)	5.86±0.12(+)	4.97±0.06(+)	4.83±0.07(+)	4.88±0.09(+)	4.69±0.07(+)	4.86±0.1(+)	5.19±0.09(+)	4.95±0.09(+)	**4.58±0.07**	3.89E-24
SN2(κ=3)	8.6±0.06(+)	9.3±0.08(+)	8.47±0.09(+)	8.35±0.08(+)	7.96±0.11(+)	6.79±0.06(+)	6.97±0.07(+)	6.87±0.09(+)	6.85±0.05(+)	**6.7±0.04**	7.90E-33
SN2(κ=4)	8.93±0.07(+)	9.53±0.09(+)	8.77±0.05(+)	8.62±0.05(+)	8.22±0.12(+)	7.09±0.06(+)	7.25±0.07(+)	7.22±0.07(+)	7.03±0.05(+)	**7.0±0.04**	4.40E-32
SN2(κ=5)	8.82±0.1(+)	9.29±0.09(+)	8.5±0.06(+)	8.36±0.07(+)	7.94±0.09(+)	6.56±0.05(+)	6.73±0.07(+)	6.67±0.08(+)	6.55±0.07(+)	**6.47±0.05**	7.61E-33
SN3(κ=3)	32.43±0.16(+)	32.04±0.06(+)	32.08±0.04(+)	31.88±0.04(+)	31.42±0.14(+)	9.5±0.25(+)	9.54±0.16(+)	9.63±0.03(+)	9.42±0.05(+)	**9.02±0.02**	3.62E-32
SN3(κ=4)	33.88±0.67(+)	31.95±0.04(+)	31.98±0.06(+)	31.79±0.05(+)	31.26±0.17(+)	12.1±0.55(+)	12.23±0.57(+)	12.94±0.06(+)	11.37±0.09(+)	**8.87±0.02**	2.93E-33
SN3(κ=5)	32.86±0.37(+)	31.98±0.04(+)	31.99±0.04(+)	31.79±0.05(+)	31.32±0.14(+)	13.19±0.46(+)	13.62±0.55(+)	14.31±0.05(+)	12.68±0.08(+)	**8.91±0.02**	1.89E-33
count(*)	0	0	0	0	0	0	0	0	0	9	

Data	GT	GA	MFEA-II	MFEA-AKT*	MFEA-BLKT*	MFEA-AAT	MFMA	MFMA-RAT	MTCMO*	MFMA-AAT	
SN1(κ=3)	3.96E-18	4.06E-39	6.00E-24	5.27E-19	1.30E-19	2.26E-06	2.17E-14	6.30E-08	6.45E-07	–	T-test *P*-value
SN1(κ=4)	8.18E-21	7.66E-34	3.45E-19	9.63E-13	4.95E-15	7.91E-05	3.77E-13	2.76E-05	3.18E-07	–	T-test *P*-value
SN1(κ=5)	6.19E-11	6.96E-33	2.46E-20	6.09E-14	1.62E-14	2.09E-05	1.03E-12	1.78E-07	1.33E-07	–	T-test *P*-value
SN2(κ=3)	2.06E-50	5.03E-51	2.10E-43	2.48E-44	6.95E-35	2.65E-06	5.60E-17	9.78E-07	1.15E-07	–	T-test *P*-value
SN2(κ=4)	2.44E-49	1.57E-50	1.98E-50	1.95E-50	2.16E-33	1.45E-06	4.93E-16	6.30E-08	2.17E-05	–	T-test *P*-value
SN2(κ=5)	2.67E-46	1.55E-50	5.10E-49	2.13E-46	3.28E-40	5.12E-06	1.13E-15	7.33E-08	1.29E-06	–	T-test *P*-value
SN3(κ=3)	1.57E-78	1.70E-94	1.13E-97	4.81E-99	5.57E-79	6.36E-10	4.68E-17	1.66E-44	1.74E-31	–	T-test *P*-value
SN3(κ=4)	1.30E-55	2.26E-98	2.29E-94	3.67E-95	2.71E-76	1.25E-25	1.09E-25	9.43E-75	7.34E-51	–	T-test *P*-value
SN3(κ=5)	8.22E-65	1.83E-98	1.60E-100	2.37E-95	4.00E-80	6.17E-33	1.26E-31	9.91E-74	9.63E-55	–	T-test *P*-value

The best performance values are shown in bold.
* indicates the corresponding algorithm is a modified version.



(a) SN3(κ=3) upper parts (b) SN3(κ=4) upper parts (c) SN3(κ=5) upper parts

(d) SN3(κ=3) lower parts (e) SN3(κ=4) lower parts (f) SN3(κ=5) lower parts

Fig. 10 Separate display of convergence trace on SN3

the use of different evaluation criteria, ensuring the efficacy of crossover operations among chromosomes with varying skill factors. Such effective crossovers are pivotal to ensuring positive transfer in EMT. Therefore, the comprehensive experimental outcomes demonstrate that MFMA-AAT, which incorporates AAT, identical fitness evaluation and memetic search, achieves a significant performance enhancement over EMT algorithms that do not employ these techniques.

6 Conclusion

Future mobile networks require ultralow user-perceived latency. An MEC-enabled solution has been introduced to address these challenges based on an MFEA paradigm. Specifically, we propose the MFMA-AAT algorithm. MFMA-AAT uses K-means to cluster UE and design positive auxiliary tasks without expert knowledge according to cluster groups, and thereafter adopts adaptive selection to obtain an effective auxiliary task among subtasks. MFMA-AAT detects a community to decode a group of feasible MEC servers to efficiently place UE profiles. In a congested latency-sensitive MEC network, MFMA-AAT can achieve considerably lower user-perceived latency than traditional algorithms and recent MFEAs. The advantage of MFMA-AAT lies in the adaptive selection of auxiliary tasks in different locations and time periods. Unlike existing EMT-based algorithms, which construct auxiliary tasks, e.g., decomposed subtasks and simplified tasks, based on prior knowledge, MFMA-AAT constructs auxiliary tasks based on the real-time physical characteristics of MEC, leading to more accurate optimization. The issue of constructing auxiliary tasks that depend heavily on prior knowledge is identified based on past studies, a novel construction strategy that is free from such dependencies is proposed.

Despite the promising performance of MFMA-AAT, this work still suffers from a few limitations. For example, the modeling assumptions are relatively simple, without taking into account the diversity of MEC network topologies. More evaluation metrics in MEC networks should be considered. An improve version of the algorithm featured by dynamic evolutionary optimization capability is more desirable to address real-world problems in MEC environment. Moreover, the current considered objectives relying solely on the average user-perceived latency as a benchmark could lead to a scenario where certain measurements are exceedingly high, contrasted by others that are strikingly low. To mitigate this disparity, a variance evaluation could be incorporated into the objective with carefully set weights. Another candidate solution is to use multi-objective optimization framework, from which we can achieve different trade-offs between average values and variances of user-perceived latency for the decision maker.

For the convenience of the reader, the source code of MFMA-AAT is provided at https://github.com/basket163/MFMA-AAT.

Acknowledgements This work was supported in part by the National Key Research and Development Program of China, under Grant 2022YFF1202104, and in part by the National Natural Science Foundation of China, under Grants 62471310, 12401585 and 62001300. (Corresponding author: Zexuan Zhu.)

Author Contributions G.L. and Z.Z. designed the algorithm, conducted experimental studies, and wrote the main manuscript text. Z.L. and L. L. prepared the discussion on experimental results. All authors reviewed the manuscript.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no competing interests.

References

1. Raza S, Wang S, Ahmed M, Anwar MR, Mirza MA, Khan WU (2022) Task offloading and resource allocation for IoV using 5G NR-V2X communication. IEEE Internet Things J 9(13):10 397-10 410
2. Ren J, Zhang D, He S, Zhang Y, Li T (2020) A survey on end-edge-cloud orchestrated network computing paradigms: transparent computing, mobile edge computing, fog computing, and cloudlet. ACM Comput Surv 52(6):125:1-125:36
3. Wang T, Zhang Y, Xiong NN, Wan S, Shen S, Huang S (2022) An effective edge-intelligent service placement technology for 5G-and-beyond industrial IoT. IEEE Trans Ind Inform 18(6):4148–4157
4. Qu G, Wu H, Li R, Jiao P (2021) DMRO: a deep meta reinforcement learning-based task offloading framework for edge-cloud computing. IEEE Trans Netw Serv Manag 18(3):3448–3459
5. Zhou Z, Liao H, Gu B, Mumtaz S, Rodriguez J (2020) Resource sharing and task offloading in IoT fog computing: a contract-learning approach. IEEE Trans Emerg Top Comput Intell 4(3):227–240
6. Xiao S, Liu C, Li K, Li K (2020) System delay optimization for mobile edge computing. Futur Gener Comput Syst 109(8):17–28
7. Zhu K, Zhou Y, Yi C, Wang R (2022) Computation resource configuration with adaptive QoS requirements for vehicular edge computing: a fluid-model based approach. IEEE Trans Intell Transp Syst 23(11):21 148-21 162
8. Ouyang T, Zhou Z, Chen X (2018) Follow me at the edge: mobility-aware dynamic service placement for mobile edge computing. IEEE J Sel Areas Commun 36(10):2333–2345
9. Liu Y, Wang X, Boudreau G, Sediq AB, Abou-zeid H (2020) Deep learning based hotspot prediction and beam management for adaptive virtual small cell in 5G networks. IEEE Trans Emerg Top Comput Intell 4(1):83–94
10. Anwar MR, Wang S, Akram MF, Raza S, Mahmood S (2022) 5G-enabled MEC: a distributed traffic steering for seamless service migration of internet of vehicles. IEEE Internet Things J 9(1):648–661
11. Li X, Chen S, Zhou Y, Chen J, Feng G (2022) Intelligent service migration based on hidden state inference for mobile edge computing. IEEE Trans Cogn Commun Netw 8(1):380–393
12. Wang H, Lv T, Lin Z, Zeng J (2022) Energy-delay minimization of task migration based on game theory in MEC-assisted vehicular networks. IEEE Trans Veh Technol 71(8):8175–8188
13. Sun L, Wang J, Lin B (2021) Task allocation strategy for MEC-enabled IIoTs via Bayesian network based evolutionary computation. IEEE Trans Ind Inform 17(5):3441–3449
14. Gupta A, Ong Y, Feng L (2016) Multifactorial evolution: toward evolutionary multitasking. IEEE Trans Evol Comput 20(3):343–357
15. Wei T, Wang S, Zhong J, Liu D, Zhang J (2022) A review on evolutionary multitask optimization: trends and challenges. IEEE Trans Evol Comput 26(5):941–960
16. Gupta A, Ong Y, Feng L (2018) Insights on transfer optimization: because experience is the best teacher. IEEE Trans Emerg Top Comput Intell 2(1):51–64
17. Tan KC, Feng L, Jiang M (2021) Evolutionary transfer optimization—a new frontier in evolutionary computation research. IEEE Comput Intell Mag 16(1):22–33
18. Gupta A, Ong Y (2019) Back to the roots: multi-x evolutionary computation. Cogn Comput 11(1):1–17
19. Bali KK, Ong Y, Gupta A, Tan PS (2020) Multifactorial evolutionary algorithm with online transfer parameter estimation: MFEA-II. IEEE Trans Evol Comput 24(1):69–83
20. Ma X, Yin J, Zhu A, Li X, Yu Y, Wang L, Qi Y, Zhu Z (2022) Enhanced multifactorial evolutionary algorithm with meme helper-tasks. IEEE Trans Cybern 52(8):7837–7851
21. Shang Q, Huang Y, Wang Y, Li M, Feng L (2022) Solving vehicle routing problem by memetic search with evolutionary multitasking. Memet Comput 14(1):31–44
22. Yang C, Ding J, Jin Y, Wang C, Chai T (2019) Multitasking multiobjective evolutionary operational indices optimization of beneficiation processes. IEEE Trans Autom Sci Eng 16(3):1046–1057
23. Qiao K, Yu K, Qu B, Liang J, Song H, Yue C, Lin H, Tan KC (2023) Dynamic auxiliary task-based evolutionary multitasking for constrained multiobjective optimization. IEEE Trans Evol Comput 27(3):642–656
24. Qiao K, Liang J, Liu Z, Yu K, Yue C, Qu B (2023) Evolutionary multitasking with global and local auxiliary tasks for constrained multi-objective optimization. IEEE/CAA J Autom Sin 10(10):1951–1964
25. Ming F, Gong W, Gao L (2023) Adaptive auxiliary task selection for multitasking-assisted constrained multi-objective optimization [feature]. IEEE Comput Intell Mag 18(2):18–30
26. Yu K, Wang L, Liang J, Wang H, Qiao K, Liang T (2024) Constraint subsets-based evolutionary multitasking for constrained multiobjective optimization. Swarm Evol Comput 86:101531
27. Das S, Biswas A (2021) Deployment of information diffusion for community detection in online social networks: a comprehensive review. IEEE Trans Comput Soc Syst 8(5):1083–1107
28. Zhu A, Wen Y (2021) Computing offloading strategy using improved genetic algorithm in mobile edge computing system. J Grid Comput 19(3):38
29. Bi J, Yuan H, Duanmu S, Zhou M, Abusorrah A (2021) Energy-optimized partial computation offloading in mobile-edge computing with genetic simulated-annealing-based particle swarm optimization. IEEE Internet Things J 8(5):3774–3785
30. Al-Habob AA, Dobre OA, Armada AG, Muhaidat S (2020) Task scheduling for mobile edge computing using genetic algorithm and conflict graphs. IEEE Trans Veh Technol 69(8):8805–8819
31. Liu J, Liu C, Wang B, Gao G, Wang S (2022) Optimized task allocation for IoT application in mobile-edge computing. IEEE Internet Things J 9(13):10 370-10 381
32. Tang H, Wu H, Zhao Y, Li R (2022) Joint computation offloading and resource allocation under task-overflowed situations in mobile-edge computing. IEEE Trans Netw Serv Manag 19(2):1539–1553

33. Wei T, Wang S, Zhong J, Liu D, Zhang J (2022) A review on evolutionary multitask optimization: trends and challenges. IEEE Trans Evol Comput 26(5):941–960
34. Zhou L, Feng L, Tan KC, Zhong J, Zhu Z, Liu K, Chen C (2021) Toward adaptive knowledge transfer in multifactorial evolutionary computation. IEEE Trans Cybern 51(5):2563–2576
35. Xu H, Qin AK, Xia S (2022) Evolutionary multitask optimization with adaptive knowledge transfer. IEEE Trans Evol Comput 26(2):290–303
36. Liang Z, Liang W, Wang Z, Ma X, Liu L, Zhu Z (2022) Multiobjective evolutionary multitasking with two-stage adaptive knowledge transfer based on population distribution. IEEE Trans Syst Man Cybern Syst 52(7):4457–4469
37. Feng L, Huang Y, Zhou L, Zhong J, Gupta A, Tang K, Tan KC (2021) Explicit evolutionary multitasking for combinatorial optimization: a case study on capacitated vehicle routing problem. IEEE Trans Cybern 51(6):3143–3156
38. Gao W, Cheng J, Gong M, Li H, Xie J (2022) Multiobjective multitasking optimization with subspace distribution alignment and decision variable transfer. IEEE Trans Emerg Top Comput Intell 6(4):818–827
39. Tang Z, Gong M, Xie Y, Li H, Qin AK (2022) Multi-task particle swarm optimization with dynamic neighbor and level-based inter-task learning. IEEE Trans Emerg Top Comput Intell 6(2):300–314
40. Jiang Y, Zhan Z-H, Tan KC, Zhang J (2024) Block-level knowledge transfer for evolutionary multitask optimization. IEEE Trans Cybern 54(1):558–571
41. Li G, Zhang Q, Wang Z (2022) Evolutionary competitive multitasking optimization. IEEE Trans Evol Comput 26(2):278–289
42. Liu S, Wang H, Peng W, Yao W (2022) A surrogate-assisted evolutionary feature selection algorithm with parallel random grouping for high-dimensional classification. IEEE Trans Evol Comput 26(5):1087–1101
43. Yang C, Chen Q, Zhu Z, Huang Z-A, Lan S, Zhu L (2024) Evolutionary multitasking for costly task offloading in mobile edge computing networks. IEEE Trans Evol Comput 28(2):338–352
44. Qiao K, Liang J, Yu K, Wang M, Qu B, Yue C, Guo Y (2023) A self-adaptive evolutionary multi-task based constrained multi-objective evolutionary algorithm. IEEE Trans Emerg Top Comput Intell 7(4):1098–1112
45. Wang H, Feng L, Jin Y, Doherty J (2021) Surrogate-assisted evolutionary multitasking for expensive minimax optimization in multiple scenarios. IEEE Comput Intell Mag 16(1):34–48
46. Yang C, Chen Q, Zhu Z, Huang Z-A, Lan S, Zhu L (2023) Evolutionary multitasking for costly task offloading in mobile-edge computing networks. IEEE Trans Evol Comput 28(2):338–352
47. Chen G, Li L, Chai Z (2024) Self-adaptive evolutionary multitasking algorithm for mobile edge computing in internet of things. IEEE Internet Things J 11:30323–30340
48. Li G, Liu L, Liang Z, Ma X, Zhu Z (2021) Memetic algorithm based on community detection for energy-efficient service migration optimization in 5G mobile edge computing. In 32nd IEEE annual international symposium on personal, indoor and mobile radio communications, PIMRC 2021, Helsinki, Finland, September 13–16, 2021. IEEE, pp 1–7
49. Santis ED, Giuseppi A, Pietrabissa A, Capponi M, Priscoli FD (2022) Satellite integration into 5G: deep reinforcement learning for network selection. Int J Autom Comput 19(2):127–137
50. Holland JH (1992) Adaptation in natural artificial systems, 2nd edn. MIT Press